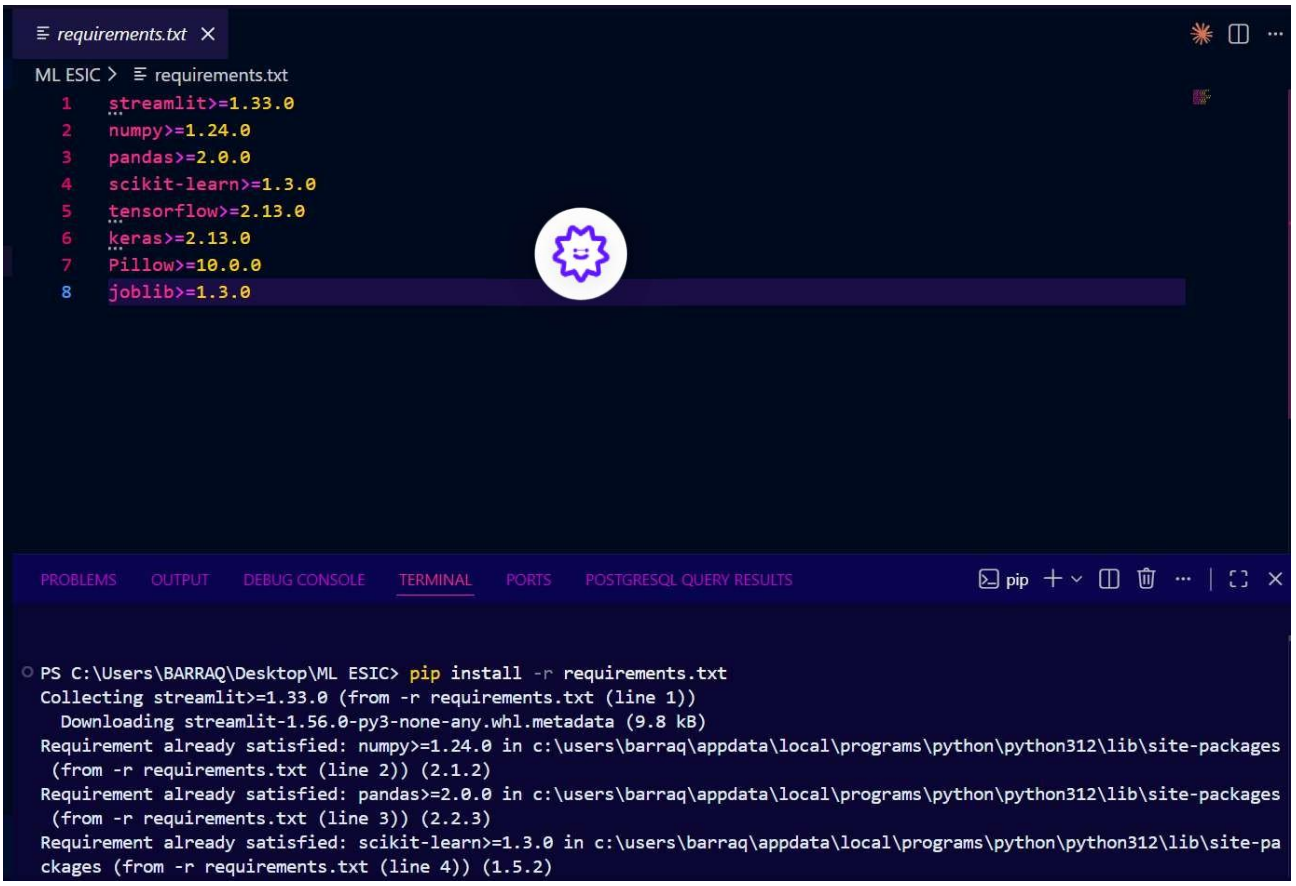
# Détection du Cancer Pulmonaire



Réalisé par : NAJIMI Sanae

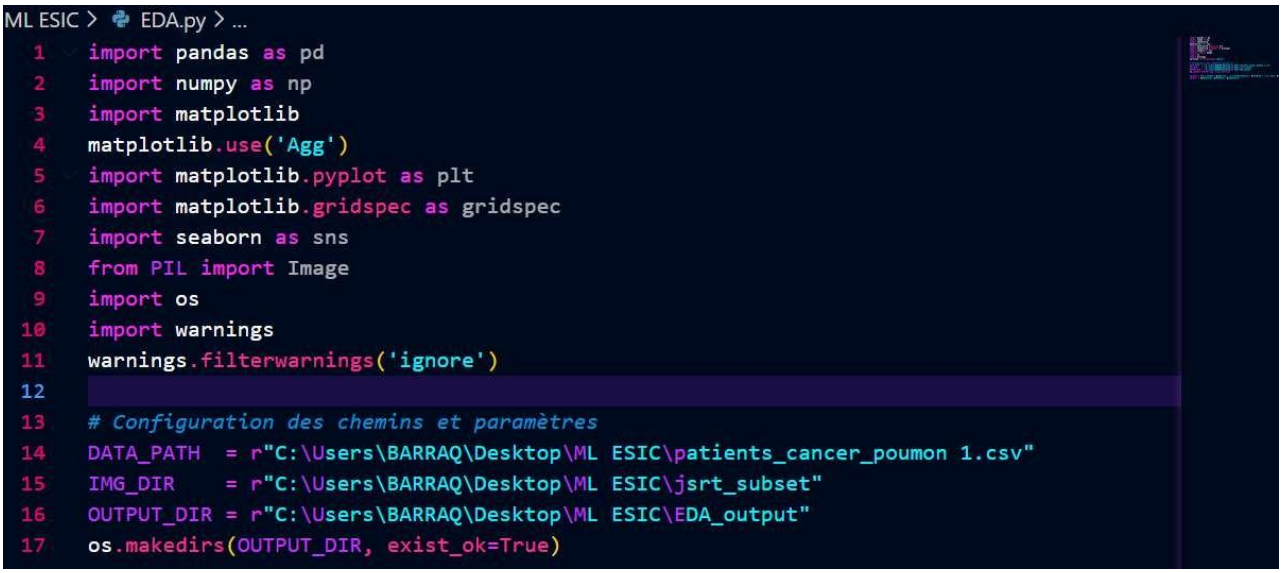
# Partie 0 : Analyse Exploratoire des Données (EDA)

## Étape 1 : Installation des bibliothèques




*La commande pip install -r requirements.txt* est utilisée pour installer en une seule instruction toutes les bibliothèques nécessaires au projet.

## Étape 2 : Importation des bibliothèques et configuration des chemins



Le script commence par l'importation de toutes les bibliothèques nécessaires à l'EDA, puis définit immédiatement les chemins vers les données.




Avant de générer les figures, on s'assure que le répertoire de sortie existe via *os.makedirs(OUTPUT_DIR, exist_ok=True)*. Le paramètre *exist_ok=True* est important car il permet d'éviter l'erreur FileExistsError si le dossier est déjà présent, rendant le script ré-exécutable sans modification ni erreur. On voit dans l'arborescence que le dossier EDA_output a bien été créé au même niveau que EDA.py, et les fichiers sources.

## Étape 3 : Chargement du dataset




Le dataset est chargé avec *pd.read_csv()* et en attribuant comme paramètre le path du fichier csv prédéfini dans les étapes précedentes.

## Étape 4 : Définition des features et des variables cibles

```python
#Définition des features et des targets
FEATURES = [
    "age", "sexe_masculin", "presence_nodule", "subtilite_nodule",
    "taille_nodule_px", "x_nodule_norm", "y_nodule_norm",
    "tabagisme_paquets_annee", "toux_chronique", "dyspnee",
    "douleur_thoracique", "perte_poids", "spo2", "antecedent_familial"
]
TARGET_TAB = "risque_malignite"
TARGET_IMG = "cancer_image"
```

Avant toute analyse, on catégorise explicitement les variables du dataset. Cette étape structure le pipeline et rend le code lisible et maintenable.

| Variable | Type | Rôle |
|---|---|---|
| FEATURES (14 variables) | Liste Python | Inputs des modèles : features cliniques et caractéristiques du nodule |
| TARGET_TAB = 'risque_malignite' | String | Cible du Modèle 1 : classification en 3 niveaux (0 / 1 / 2) |
| TARGET_IMG = 'cancer_image' | String | Cible du Modèle 2 : classification binaire (0=non cancer, 1=cancer) |

**Pourquoi deux variables cibles distinctes ?**

risque_malignite est une évaluation clinique en 3 niveaux basée sur les données tabulaires. cancer_image est le diagnostic radiologique binaire. Ces deux cibles sont corrélées ( r = 0.90) mais non identiques : le modèle d'images accède à des informations visuelles que les données tabulaires ne capturent pas. Cette séparation justifie l'architecture à deux modèles interconnectés de la Partie 2.

## Étape 5 : Mapping des données

```python
label_map = {0: "0 - Faible", 1: "1 - Intermédiaire", 2: "2 - Élevé"}
df["risque_label"] = df[TARGET_TAB].map(label_map)
```

On crée une colonne risque_label en mappant les codes numériques (0, 1, 2) vers leurs libellés textuels lisibles.. La méthode *.map()* de pandas applique le dictionnaire label_map sur toute la colonne en une seule opération vectorisée, sans boucle for donc plus efficace et rapide.

## Étape 6 : Aperçu du dataset, valeurs manquantes et distribution des cibles

```
print(" aperçu du dataset:")
print(f"  Lignes : {df.shape[0]}  |  Colonnes : {df.shape[1]}")
print(f"  Valeurs manquantes : {df.isnull().sum().sum()}")
print()

print(" Distribution de la cible (risque_malignite) :")
print(df[TARGET_TAB].value_counts().sort_index()
      .rename({0:"Faible (0)", 1:"Intermédiaire (1)", 2:"Élevé (2)"}).to_string())
print()
print(" Distribution cible image (cancer_image) :")
print(df[TARGET_IMG].value_counts().rename({0:"Non cancer", 1:"Cancer"}).to_string())
print()
print(" Statistiques descriptives :")
print(df[FEATURES].describe().round(2)
```

C'est le premier audit complet du dataset. On vérifie simultanément la taille, la complétude et la distribution des variables cibles, trois informations indispensables pour toute décision de modélisation.

```
Aperçu du dataset:

  Lignes : 184  |  Colonnes : 21
  Valeurs manquantes : 0

Distribution de la cible (risque_malignite) :
risque_malignite
Faible (0)           30
Intermédiaire (1)    54
Élevé (2)           100

Distribution cible image (cancer_image) :
cancer_image
Cancer        100
Non cancer     84
```

| Indicateur | Valeur observée | Interprétation |
|---|---|---|
| Lignes | 184 | Dataset de taille modeste mais réaliste pour une étude clinique |
| Colonnes | 21 | 20 features exploitables + 1 colonne image_path non utilisée en ML |
| Valeurs manquantes | 0 | Dataset complet aucune valeur manquante |
| risque_malignite : Faible (0) | 30 patients : 16.3% | Classe minoritaire |
| risque_malignite : Intermédiaire (1) | 54 patients : 29.3% | Classe intermédiaire |
| risque_malignite : Élevé (2) | 100 patients : 54.3% | Classe majoritaire |
| cancer_image : Cancer (1) | 100 patients : 54.3% | Classes quasi-équilibrées donc favorable pour le CNN |
| cancer_image : Non cancer (0) | 84 patients : 45.7% | Pas de rééquilibrage nécessaire pour la cible image |

**NB :** Le ratio entre classe minoritaire (Faible : 30 patients) et majoritaire (Élevé : 100 patients) est de 1:3.3. Un modèle qui prédirait toujours 'Élevé' obtiendrait 54.3% d'accuracy sans rien apprendre. Donc la Stratégie adoptée en Partie 1 est *class_weight='balanced'* dans tous les algorithmes sklearn afin d'équilibrer les paramètres du modèle et avoir une prédiction plus efficace

## Étape 7: l'analyse descriptive

```
Statistiques descriptives :
          age  sexe_masculin  presence_nodule  subtilite_nodule  taille_nodule_px  x_nodule_norm  y_nodule_norm  tabagisme
_paquets_annee  toux_chronique  dyspnee  douleur_thoracique  perte_poids    spo2  antecedent_familial
count  184.00         184.00          184.00             184.00           184.00         184.00         184.00
       184.00          184.00   184.00              184.00       184.00  184.00              184.00
mean    59.33           0.46             0.84               2.58              0.84           0.38           0.35
        30.21            0.84     0.54                0.54         0.54   93.86                0.32
std     12.77           0.50             0.37               1.29              0.37           0.28           0.22
         7.77            0.37     0.50                0.50         0.50    2.25                0.47
min     21.00           0.00             0.00               1.00              0.00           0.00           0.00
         9.00            0.00     0.00                0.00         0.00   92.00                0.00
25%     51.00           0.00             1.00               1.00              1.00           0.21           0.20
        24.50            1.00     0.00                0.00         0.00   92.00                0.00
50%     61.00           0.00             1.00               3.00              1.00           0.29           0.36
        31.40            1.00     1.00                0.00         1.00   92.00                0.00
75%     69.00           1.00             1.00               4.00              1.00           0.70           0.52
        36.17            1.00     1.00                1.00         1.00   95.00                1.00
max     80.00           1.00             1.00               5.00              1.00           0.87           0.83
        43.00            1.00     1.00                1.00         1.00   98.00                1.00
 *  Terminal will be reused by tasks, press any key to close it.
```

Intérprétation des résultats :

| Variable | Moyenne | Std | Min – Max | Observation diagnostique |
|---|---|---|---|---|
| age | 59.3 | 12.8 | 21 – 80 | Population adulte à risque : âge médian 61 ans, typique du cancer pulmonaire |
| tabagisme_paquets_annee | 30.2 | 7.8 | 9 – 43 | Exposition élevée et relativement homogène |
| spo2 (%) | 93.9 | 2.3 | 92 – 98 | Médiane à 92% : Cliniquement, une SpO2 < 95% signale une hypoxémie légère |
| subtilite_nodule | 2.58 | 1.29 | 1 – 5 | Variable ordinale étalée sur toute sa plage : discriminante potentielle |
| presence_nodule | 0.84 | 0.37 | 0 – 1 | 84% des patients ont un nodule détecté à l'imagerie |
| sexe_masculin | 0.46 | 0.50 | 0 – 1 | Légèrement plus de femmes (54%) dans la cohorte |
| antecedent_familial | 0.32 | 0.47 | 0 – 1 | 32% des patients ont un antécédent familial de cancer du poumon |

# Visualisations :

```
# FIGURE 1: DISTRIBUTION DES CIBLES & ÂGE PAR RISQUE
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle("Figure 1 — Distribution des cibles et profil démographique",
             fontsize=14, fontweight="bold", y=1.02)
# 1a. Distribution du risque de malignité
counts = df[TARGET_TAB].value_counts().sort_index()
bars = axes[0].bar(
    ["Faible (0)", "Intermédiaire (1)", "Élevé (2)"],
    counts.values, color=COLORS, edgecolor="white", linewidth=1.5
)
for bar, val in zip(bars, counts.values):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
                 f"{val}\n({val/len(df)*100:.0f}%)",
                 ha="center", va="bottom", fontsize=10, fontweight="bold")
axes[0].set_title("Distribution du risque de malignité", fontsize=12)
axes[0].set_ylabel("Nombre de patients")
axes[0].set_ylim(0, max(counts.values) * 1.2)
axes[0].spines[["top","right"]].set_visible(False)
# 1b. Répartition binaire cancer_image
img_counts = df[TARGET_IMG].value_counts()
wedges, texts, autotexts = axes[1].pie(
    img_counts.values,
    labels=["Non cancer", "Cancer"],
    colors=["#3498db", "#e74c3c"],
    autopct="%1.1f%%",
    startangle=90,
    wedgeprops={"edgecolor": "white", "linewidth": 2}
)
for t in autotexts:
    t.set_fontweight("bold")
axes[1].set_title("Répartition cible image (cancer_image)", fontsize=12)
# 1c. Distribution de l'âge par niveau de risque
```

```
for i, (code, label) in enumerate(label_map.items()):
    subset = df[df[TARGET_TAB] == code]["age"]
    axes[2].hist(subset, bins=12, alpha=0.65, label=label, color=COLORS[i], edgecolor="white")
axes[2].set_title("Distribution de l'âge par risque", fontsize=12)
axes[2].set_xlabel("Âge (années)")
axes[2].set_ylabel("Fréquence")
axes[2].legend()
axes[2].spines[["top","right"]].set_visible(False)
plt.tight_layout()
fig.savefig(f"{OUTPUT_DIR}/fig1_distribution_cibles.png", dpi=150, bbox_inches="tight")
plt.close()

# FIGURE 2 : VARIABLES CLINIQUES PAR RISQUE

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle("Figure 2 — Variables cliniques selon le niveau de risque",
             fontsize=14, fontweight="bold")
# Définition des variables à visualiser
plot_configs = [
    ("tabagisme_paquets_annee", "Tabagisme (paquets/an)", "boxplot"),
    ("spo2",                    "SpO₂ (%)",               "boxplot"),
    ("taille_nodule_px",        "Taille nodule (px)",     "boxplot"),
    ("toux_chronique",          "Toux chronique",         "barplot"),
    ("dyspnee",                 "Dyspnée",                "barplot"),
    ("antecedent_familial",     "Antécédent familial",    "barplot"),
]
for ax, (col, title, kind) in zip(axes.flat, plot_configs):
    if kind == "boxplot":
        data_grouped = [df[df[TARGET_TAB] == k][col].values for k in [0,1,2]]
        bp = ax.boxplot(data_grouped, patch_artist=True,
                        medianprops={"color":"black","linewidth":2})
        for patch, color in zip(bp["boxes"], COLORS):
            patch.set_facecolor(color)
            patch.set_alpha(0.7)
```

```
        ax.set_xticks([1,2,3])
        ax.set_xticklabels(["Faible", "Interm.", "Élevé"])
    else:  # barplot de proportions
        props = df.groupby(TARGET_TAB)[col].mean() * 100
        bars = ax.bar(["Faible", "Interm.", "Élevé"], props.values,
                      color=COLORS, edgecolor="white", linewidth=1.5, alpha=0.85)
        for bar, val in zip(bars, props.values):
            ax.text(bar.get_x() + bar.get_width()/2, b      eight() + 1,
                    f"{val:.0f}%", ha="center", va="bottom", fontsize=9)
        ax.set_ylabel("% de patients")
        ax.set_ylim(0, 115)
    ax.set_title(title, fontsize=11, fontweight="bold")
    ax.spines[["top","right"]].set_visible(False)

plt.tight_layout()
fig.savefig(f"{OUTPUT_DIR}/fig2_variables_cliniques.png", dpi=150, bbox_inches="tight")
plt.close()

# FIGURE 3 : MATRICE DE CORRÉLATION

fig, ax = plt.subplots(figsize=(13, 10))

corr_cols = FEATURES + [TARGET_TAB, TARGET_IMG]
corr = df[corr_cols].corr()

mask = np.triu(np.ones_like(corr, dtype=bool))  # masque triangle sup
sns.heatmap(
    corr, mask=mask, annot=True, fmt=".2f", cmap="RdYlGn",
    center=0, vmin=-1, vmax=1,
    linewidths=0.5, linecolor="white",
    cbar_kws={"shrink": 0.8},
    ax=ax
)
ax.set_title("Figure 3 — Matrice de corrélation (features + cibles)",
```

```
            fontsize=13, fontweight="bold", pad=15)
plt.tight_layout()
fig.savefig(f"{OUTPUT_DIR}/fig3_correlation.png", dpi=150, bbox_inches="tight")
plt.close()



# FIGURE 4 : RADIOS THORACIQUES PAR CLASSE

classes = {"sain": "Sain", "benin": "Bénin", "malin": "Malin"}
n_per_class = 3
fig = plt.figure(figsize=(14, 6))
fig.suptitle("Figure 4 — Radios thoraciques représentatives par classe",
             fontsize=14, fontweight="bold")
col_idx = 0
for classe, label in classes.items():
    class_dir = os.path.join(IMG_DIR, classe)
    images = sorted(os.listdir(class_dir))[:n_per_class]
    for row_idx, img_name in enumerate(images):
        ax = fig.add_subplot(n_per_class, 3, row_idx * 3 + col_idx + 1)
        img = Image.open(os.path.join(class_dir, img_name)).convert("L")
        ax.imshow(img, cmap="gray")
        ax.axis("off")
        if row_idx == 0:
            color = {"sain":"#2ecc71","benin":"#f39c12","malin":"#e74c3c"}[classe]
            ax.set_title(f" {label}" if classe=="sain" else
                         f" {label}" if classe=="benin" else f" {label}",
                         fontsize=12, fontweight="bold", color=color)
    col_idx += 1
plt.tight_layout()
fig.savefig(f"{OUTPUT_DIR}/fig4_radios_par_classe.png", dpi=150, bbox_inches="tight")
plt.close()
```

Quatre figures ont été produites pour explorer visuellement les données. Elles couvrent la distribution des cibles, les relations entre variables cliniques et niveau de risque, les corrélations entre features, et les caractéristiques visuelles des radios thoraciques par classe

## Figure 1: Distribution des cibles et profil démographique

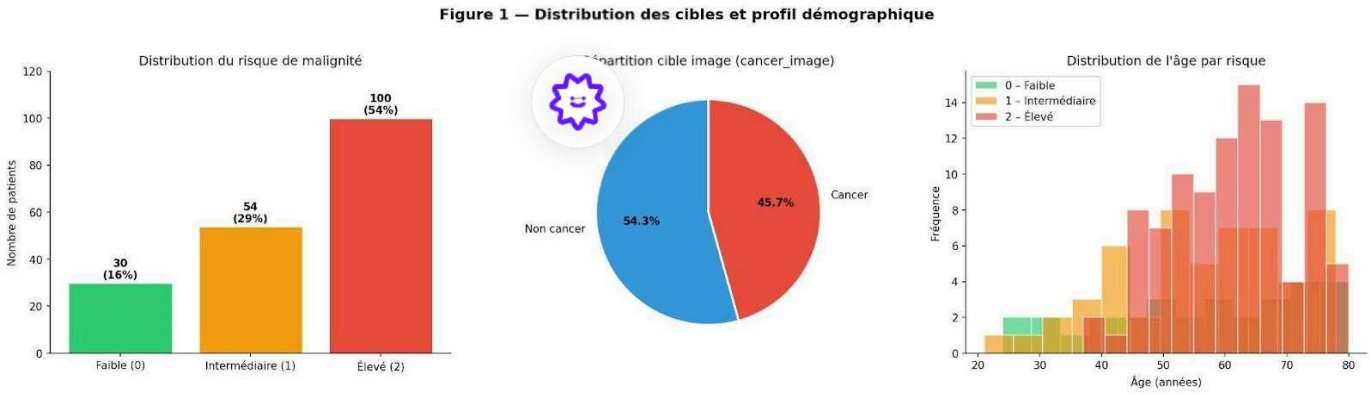



Figure 1 : Distribution du risque de malignité, répartition cancer_image, et distribution de l'âge par niveau de risque

- Graphe Distribution du risque de malignité : Déséquilibre marqué, la classe 'Élevé (2)' représente 54% des patients. Ce constat valide l'utilisation de class_weight='balanced' en Partie 1 pour ne pas biaiser le modèle vers la classe majoritaire.
- Graphe Répartition cible image : La cible image cancer_image est quasi-équilibrée (54.3% cancer vs 45.7% non-cancer).
- Graphe Distribution de l'âge par risque : Le risque élevé (rouge) se concentre dans les tranches 55–80 ans, cohérent avec l'épidémiologie réelle. Le risque faible (vert) est présent dès 20 ans mais marginal. L'âge seul a une corrélation faible (r=0.19): il contribue mais ne suffit pas à prédire.

## Figure 2: Variables cliniques selon le niveau de risque

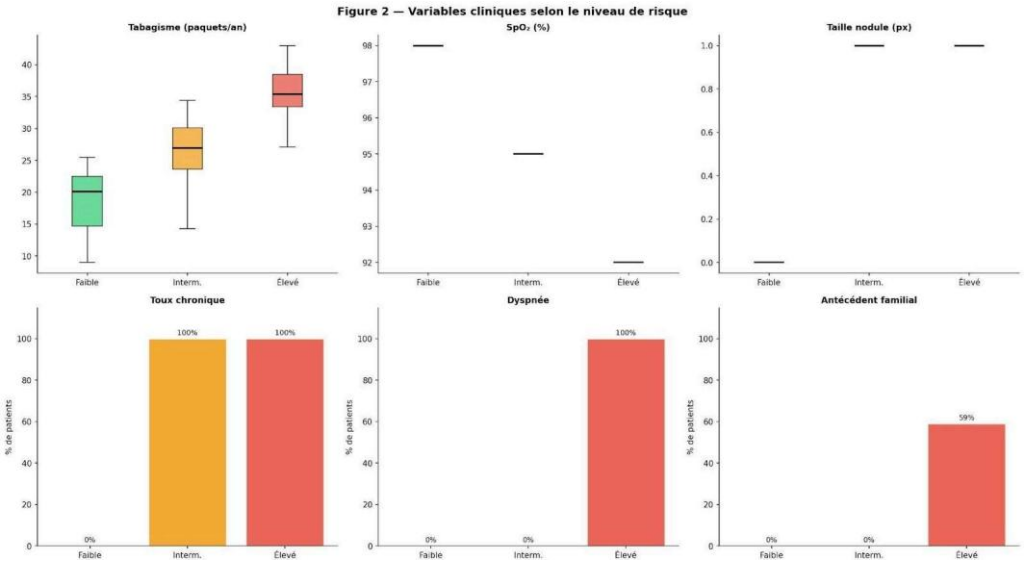


Figure 2: Boxplots (tabagisme, SpO2, taille nodule) et barplots de proportion (toux, dyspnée, antécédent familial) par niveau de risque

- Tabagisme (paquets/an) : Progression très nette entre les groupes: médiane ~20 pour risque faible contre ~36 pour risque élevé. C'est la variable continue la plus discriminante visuellement (r=0.85 avec la cible).
- SpO2 (%) : Inversion parfaite: les patients à risque élevé ont une SpO2 médiane de 92% contre 98% pour le groupe faible. Anti-corrélation quasi-parfaite avec la cible (r=-1.00), confirmant son rôle de prédicteur dominant.
- Taille nodule : Variable binaire: 100% des patients à risque intermédiaire et élevé ont un nodule détecté, contre 0% pour le groupe faible. Séparateur parfait pour la classe 0 vs les autres.
- Toux chronique & Dyspnée : 100% des patients à risque intermédiaire et élevé présentent une toux chronique. La dyspnée est exclusive au groupe élevé (100%). Ces valeurs parfaites soulèvent un risque de data leakage discuté en Partie 3.
- Antécédent familial : 59% des patients à risque élevé ont un antécédent familial, contre 0% pour les groupes faible et intermédiaire. Facteur génétique discriminant uniquement pour la classe la plus sévère.

## Figure 3: Matrice de corrélation

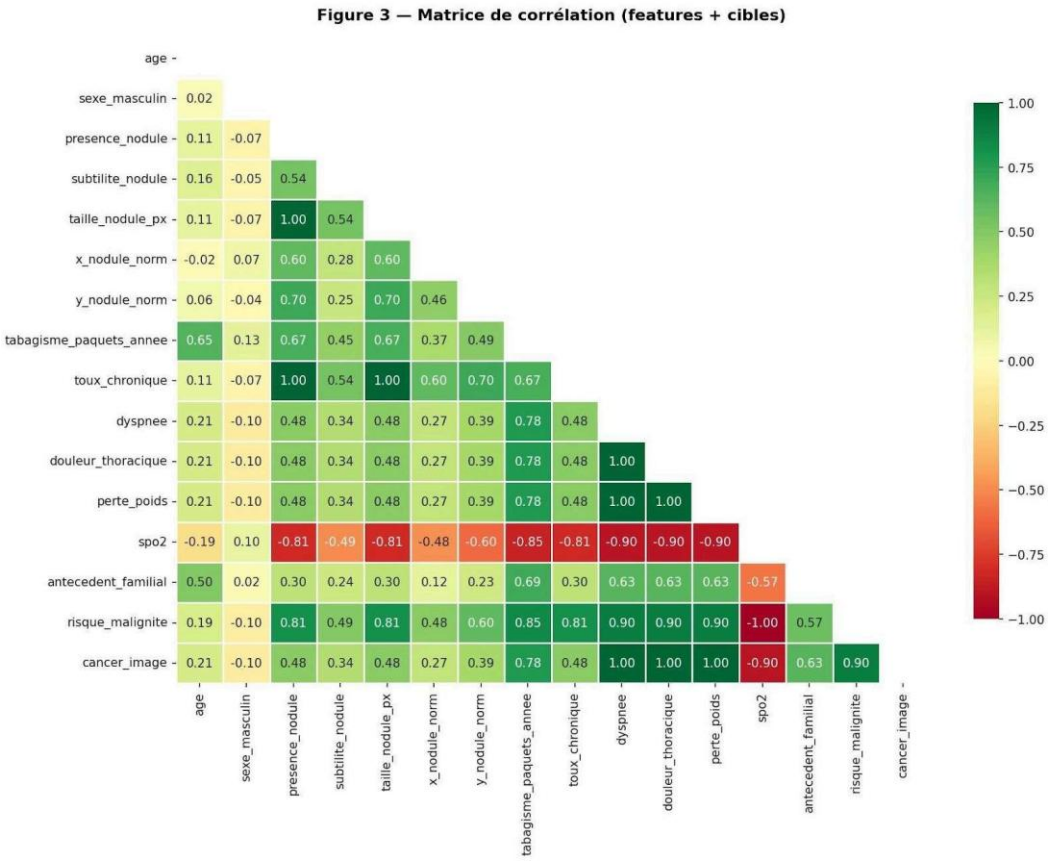



Figure 3: Matrice de corrélation de Pearson (features + cibles)

La matrice de corrélation (coefficient de Pearson) quantifie les relations linéaires entre toutes les variables numériques. Les corrélations les plus fortes avec risque_malignite sont :

| Feature | Corr. risque_malignite | Corr. cancer_image | Interprétation |
|---|---|---|---|
| spo2 | -1.00 | -0.90 | Anti-corrélation parfaite: variable quasi-déterministe de la cible tabulaire |
| douleur_thoracique | +0.90 | +1.00 | Symptôme directement lié au diagnostic |
| perte_poids | +0.90 | +1.00 | Signal fort |
| dyspnee | +0.90 | +1.00 | 100% des cas élevés |
| tabagisme_paquets_annee | +0.85 | +0.78 | Variable de risque authentique : relation dose-réponse réaliste |
| toux_chronique | +0.81 | +0.48 | Symptôme fort : corrélation moindre avec la cible image |
| age | +0.19 | +0.21 | Corrélation faible : l'âge seul n'est pas prédictif ici |
| sexe_masculin | -0.10 | -0.10 | Quasi-nulle: le sexe n'est pas discriminant dans cette cohorte |

**NB :** risque_malignite et cancer_image sont corrélées à 0.90 entre elles. Cette forte corrélation inter-cibles confirme la cohérence du dataset et justifie que les probabilités du Modèle 1 seront un apport pertinent pour le Modèle 2 dans l'approche multimodale de la Partie 2.

## Figure 4: Radios thoraciques par classe

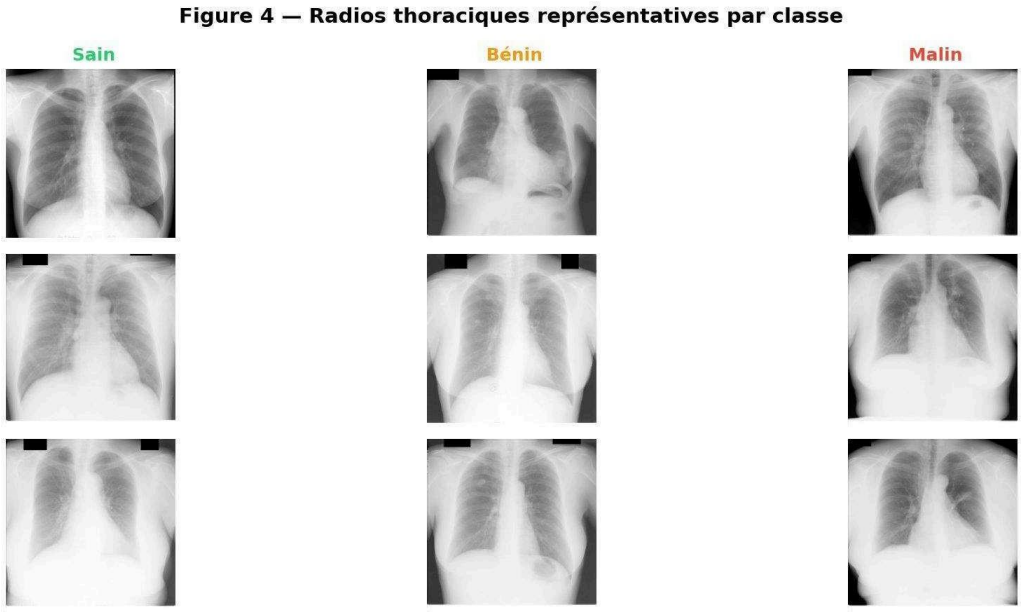



Figure 4: Exemples de radios thoraciques par classe : Sain (30 images) / Bénin (54 images) / Malin (100 images)

| Classe | Effectif | Description visuelle |
|---|---|---|
| Sain | 30 images | Champs pulmonaires clairs, pas d'opacité anormale, trame vasculaire normale |
| Bénin | 54 images | Opacité localisée avec bords nets, masse bien délimitée |
| Malin | 100 images | Masse dense aux bords irréguliers, parfois volumineuse |

# Partie 1: Modèle 1, Classif ..on Tabulaire

## 1.1 Imports scikit-learn et configuration

Avant tout entraînement, on importe sklearn ainsi que XGBoost. Les chemins vers les données et les dossiers de sortie sont centralisés dans trois variables de configuration.

```
11
12  from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
13  from sklearn.preprocessing import StandardScaler, label_binarize
14  from sklearn.pipeline import Pipeline
15  from sklearn.ensemble import RandomForestClassifier
16  from sklearn.linear_model import LogisticRegression
17  from sklearn.metrics import (classification_report, confusion_matrix,
18                               f1_score, accuracy_score, roc_auc_score)
19  from xgboost import XGBClassifier
20
21  # CONFIG
22  DATA_PATH  = r"C:\Users\BARRAQ\Desktop\ML ESIC\patients_cancer_poumon 1.csv"
23  OUTPUT_DIR = r"C:\Users\BARRAQ\Desktop\ML ESIC\Partie1_output"
24  MODEL_DIR  = r"C:\Users\BARRAQ\Desktop\ML ESIC\models"
25  os.makedirs(OUTPUT_DIR, exist_ok=True)
26  os.makedirs(MODEL_DIR, exist_ok=True)
27
```

## 1.2 Chargement du dataset et split stratifié

Le dataset est chargé et immédiatement divisé en jeu d'entraînement (80%) et jeu de test (20%). Le paramètre stratify=y garantit que chaque fraction reproduit exactement les proportions de classes du dataset complet.

```
# 1. CHARGEMENT & SPLIT

df = pd.read_csv(DATA_PATH)
X  = df[FEATURES]
y  = df[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED, stratify=y
)
print("SPLIT TRAIN / TEST")
print(f"  Train : {X_train.shape[0]} patients")
print(f"  Test  : {X_test.shape[0]} patients")
print(f"  Distribution train : {dict(y_train.value_counts().sort_index())}")
print(f"  Distribution test  : {dict(y_test.value_counts().sort_index())}")
```

```
Desktop/ML ESIC/Classification_tab.py"
SPLIT TRAIN / TEST
  Train : 147 patients
  Test  : 37 patients
  Distribution train : {0: np.int64(24), 1: np.int64(43), 2: np.int64(80)}
  Distribution test  : {0: np.int64(6), 1: np.int64(11), 2: np.int64(20)}
PS C:\Users\BARRAQ\Desktop\ML ESIC>
```

| Partition | Effectif | Faible (0) | Intermédiaire (1) | Élevé (2) |
|---|---|---|---|---|
| Train (80%) | 147 patients | 24 | 43 | 80 |
| Test (20%) | 37 patients | 6 | 11 | 20 |
| Ratio train/test | 80/20 | 16.3 | 29.3% | 54.4% |

## 1.3 Définition des 3 modèles

Chaque algorithme est encapsulé dans un Pipeline sklearn. Cette architecture uniformise l'interface d'entraînement et d'évaluation : tous les modèles exposent les mêmes méthodes *fit()*, *predict()* et *predict_proba()*, quel que soit leur degré de complexité interne.

```
# 2. DÉFINITION DES 3 MODÈLES (pipelines)

models = {
  "Random Forest": Pipeline([
    ("clf", RandomForestClassifier(
      n_estimators=200,
      max_depth=None,
      class_weight="balanced",
      random_state=SEED
    ))
  ]),
  "XGBoost": Pipeline([
    ("clf", XGBClassifier(
      n_estimators=200,
      learning_rate=0.1,
      max_depth=5,
      use_label_encoder=False,
      eval_metric="mlogloss",
      random_state=SEED
    ))
```

```
    ]),
    "Régression Logistique": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(
            max_iter=1000,
            class_weight="balanced",
            multi_class="multinomial",
            solver="lbfgs",
            random_state=SEED
        ))
    ]),
}
```

| Modèle | Hyperparamètres clés | Rationalité du choix |
|---|---|---|
| Random Forest | n_estimators=200, max_depth=None, class_weight='balanced' | 200 arbres pour stabiliser la variance. max_depth=None laisse chaque arbre se développer pleinement, la forêt elle-même régularise via le vote. class_weight='balanced' compense le déséquilibre 30/54/100. |
| XGBoost | n_estimators=200, lr=0.1, max_depth=5, eval_metric='mlogloss' | learning_rate=0.1 avec 200 estimateurs : compromis convergence/généralisation standard. max_depth=5 limite la complexité des arbres faibles pour éviter l'overfitting. |
| Rég. Logistique | StandardScaler + max_iter=1000, multinomial, solver='lbfgs', class_weight='balanced' | Scaler obligatoire — LR est sensible aux échelles. solver='lbfgs' gère nativement le multinomial. max_iter=1000 pour assurer la convergence sur des données avec variables quasi-parfaitement séparées. |

## 1.4 Résultats d'entraînement et évaluation

Les trois modèles sont entraînés puis évalués sur le jeu de test. La validation croisée 5-fold sur le jeu d'entraînement permet d'estimer la performance hors-échantillon sans contaminer le test.

```python
# 4. COMPARAISON & SÉLECTION DU MEILLEUR MODÈLE

print("COMPARAISON DES MODÈLES")
comparison = pd.DataFrame({
    name: {
        "CV F1-macro": f"{r['cv_mean']:.4f} ± {r['cv_std']:.4f}",
        "Test Accuracy": f"{r['accuracy']:.4f}",
        "Test F1-macro": f"{r['f1_macro']:.4f}",
        "AUC-ROC":       f"{r['auc']:.4f}",
    }
    for name, r in results.items()
}).T
print(comparison.to_string())

best_name = max(results, key=lambda k: results[k]["f1_macro"])
best      = results[best_name]
print(f"\n  Meilleur modèle : {best_name}  (F1-macro test = {best['f1_macro']:.4f})")
```

Les rapports de classification complets confirment l'uniformité des performances sur les 37 patients du jeu de test. Chaque classe est classée avec une précision, un recall et un F1-score de 1.00, sans exception.

**Random Forest :**

```
── Random Forest ──
  CV F1-macro (5-fold) : 1.0000 ± 0.0000
  Accuracy  : 1.0000
  F1 macro  : 1.0000
  AUC-ROC   : 1.0000

                  precision    recall  f1-score   support

       Faible (0)       1.00      1.00      1.00         6
Intermédiaire (1)       1.00      1.00      1.00        11
        Élevé (2)       1.00      1.00      1.00        20

         accuracy                           1.00        37
        macro avg       1.00      1.00      1.00        37
     weighted avg       1.00      1.00      1.00        37
```

**XGboost :**

```
── XGBoost ──
  CV F1-macro (5-fold) : 1.0000 ± 0.0000
  Accuracy  : 1.0000
  F1 macro  : 1.0000
  AUC-ROC   : 1.0000

                  precision    recall  f1-score   support

       Faible (0)       1.00      1.00      1.00         6
Intermédiaire (1)       1.00      1.00      1.00        11
        Élevé (2)       1.00      1.00      1.00        20

         accuracy                           1.00        37
        macro avg       1.00      1.00      1.00        37
     weighted avg       1.00      1.00      1.00        37
```

**Régression logistique :**

```
── Régression Logistique ──
  CV F1-macro (5-fold) : 1.0000 ± 0.0000
  Accuracy   : 1.0000
  F1 macro   : 1.0000
  AUC-ROC    : 1.0000

                  precision    recall  f1-score   support

       Faible (0)       1.00      1.00      1.00         6
Intermédiaire (1)       1.00      1.00      1.00        11
       Élevé (2)        1.00      1.00      1.00        20

        accuracy                           1.00        37
       macro avg       1.00      1.00      1.00        37
    weighted avg       1.00      1.00      1.00        37
```

Synthèse du modèle choisi :

```
COMPARAISON DES MODÈLES
                      CV F1-macro Test Accuracy Test F1-macro AUC-ROC
Random Forest       1.0000 ± 0.0000       1.0000        1.0000  1.0000
XGBoost             1.0000 ± 0.0000       1.0000        1.0000  1.0000
Régression Logistique  1.0000 ± 0.0000       1.0000        1.0000  1.0000

  Meilleur modèle : Random Forest  (F1-macro test = 1.0000)
```

Les trois modèles atteignant des performances identiques (F1-macro = 1.0000), la sélection du Random Forest repose sur des critères qualitatifs : robustesse au déséquilibre de classes via class_weight='balanced', absence de dépendance à la normalisation, interprétabilité native des features via feature_importances_, et stabilité parfaite en cross-validation (σ = 0.0000). En contexte médical, ces propriétés sont des critères de choix aussi importants que la performance brute, un modèle déployable est un modèle compréhensible et auditable.

## 1.5 Visualisation des performances

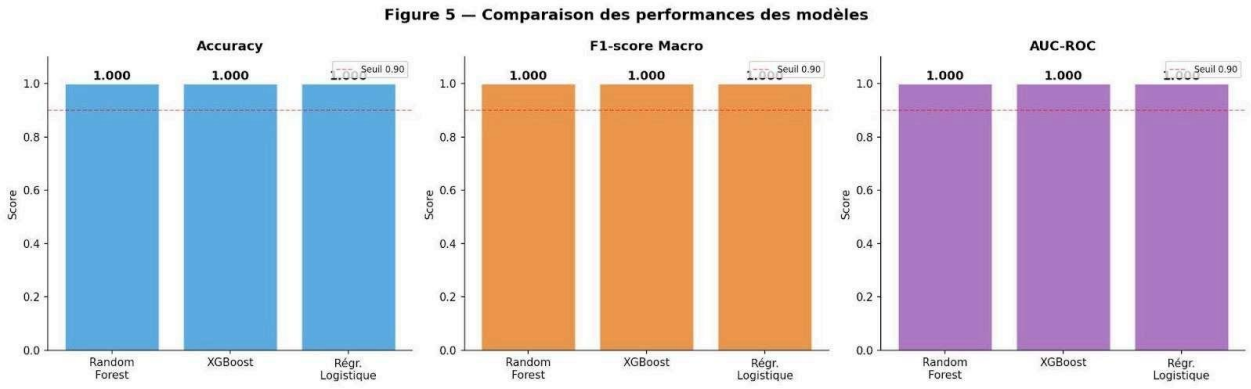



Figure 5 : Comparaison des performances des 3 modèles (Accuracy, F1-macro, AUC-ROC)

Les trois barplots confirment visuellement l'équivalence parfaite des trois algorithmes sur ce dataset. Tous atteignent le plafond théorique de 1.000 sur les trois métriques, s'élevant bien au-dessus du seuil de référence à 0.90 tracé en rouge. Cette uniformité est elle-même un signal anormal : dans des conditions réelles, les trois algorithmes auraient des profils de performance différenciés reflétant leurs biais inductifs respectifs. L'absence de différenciation pointe vers des features trop facilement séparables — confirmant le leakage.

## 1.6 Matrices de confusion

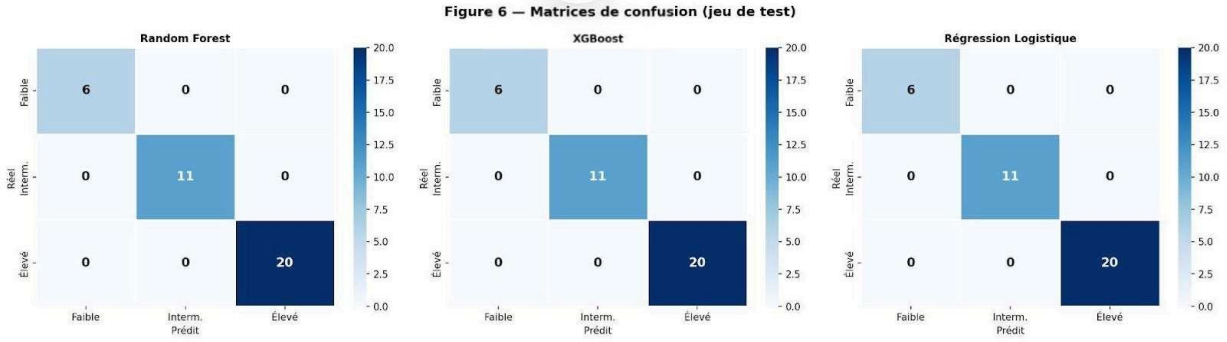



Figure 6 :Matrices de confusion des 3 modèles sur le jeu de test (37 patients)

Les trois matrices sont parfaitement diagonales : chaque patient est classé dans sa classe réelle sans exception. Sur 37 patients de test, 6 Faible, 11 Intermédiaire, 20 Élevé, zéro erreur de classification. Ce résultat mérite une lecture à deux niveaux :

- De ce qui parait : les trois modèles sont parfaits et prêts pour la production médicale. Mais :

  Des frontières de décision aussi nettes sur un dataset de 184 patients révèlent que le problème est trivialement linéaire dans l'espace des features, ce qui n'est possible que si certaines features encodent directement la cible. En pratique clinique, un AUC de 1.00 sur un problème de diagnostic n'arrive jamais sans fuite d'information.

## 1.7 Importance des features

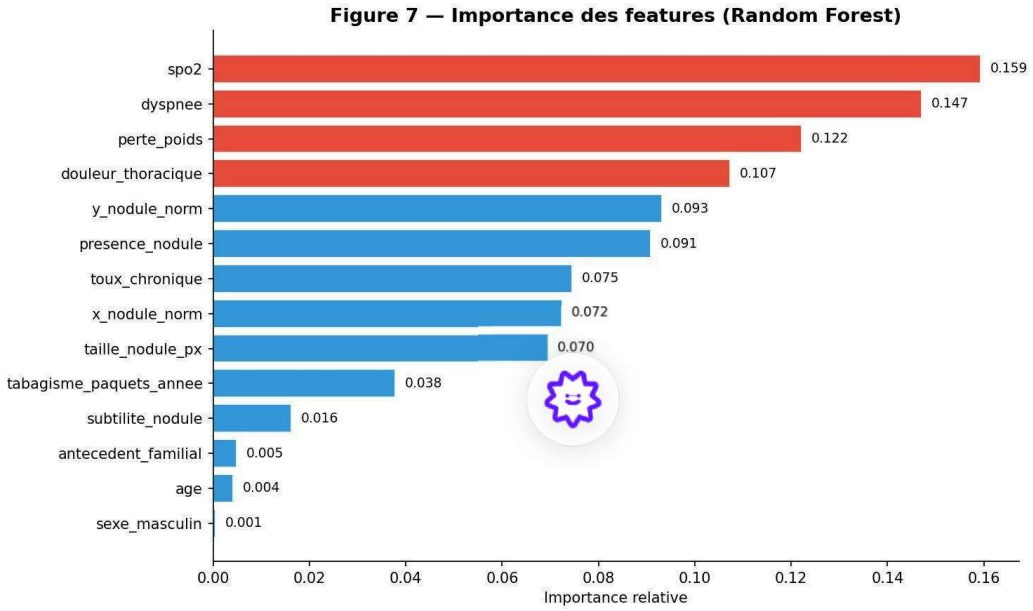



Figure 7 : Importance des features selon le Random Forest (meilleur modèle sélectionné)

Le graphique révèle une hiérarchie de features en deux groupes distincts, séparés par une rupture nette autour du seuil de 10% d'importance :

**Groupe rouge: Features dominantes (importance > 10%)**

- spo2: 15.9% : La saturation en oxygène est la variable la plus prédictive. Biologiquement cohérent : l'hypoxie est une conséquence directe de la masse tumorale qui réduit la surface alvéolaire effective. C'est aussi la variable la plus suspecte de leakage, sa valeur à 92% médiane est anormalement basse même pour les cas 'non cancer'.
- dyspnee: 14.7% : L'essoufflement est un symptôme paraneoplasique tardif. Sa présence à 100% dans le groupe élevé et 0% dans le groupe faible en fait un séparateur binaire parfait, comportement caractéristique d'une variable qui encode la cible.
- perte_poids: 12.2% : La cachexie cancéreuse est un marqueur systémique, pas radiologique. Sa présence dans les features tabulaires d'un dataset d'imagerie médicale est cliniquement douteuse.
- douleur_thoracique: 10.7% : Même profil que dyspnee, 0%/100% entre les groupes. Troisième séparateur parfait.

**Groupe bleu: Features secondaires (importance < 10%)**

- y_nodule_norm (9.3%) et presence_nodule (9.1%) : La localisation verticale et la présence du nodule sont les features radiologiques les plus informatives, celles qui reflètent réellement l'information visuelle des radios.
- toux_chronique (7.5%), x_nodule_norm (7.2%), taille_nodule_px (7.0%) : Contribution intermédiaire.

- tabagisme_paquets_annee (3.8%) : Surprenant compte tenu de sa forte corrélation avec la cible en EDA. Le Random Forest lui accorde peu d'importance car les variables binaires ci-dessus suffisent à séparer les classes, le tabagisme devient redondant.
- age (0.4%), sexe_masculin (0.1%) : Quasi-nuls, confirmant que les facteurs démographiques ne sont pas discriminants dans ce dataset structuré.

## 1.8 Les probabilités de prédiction par classe.

Le Random Forest ne prédit pas uniquement une classe, il produit pour chaque patient un vecteur de 3 probabilités reflétant le degré de confiance [illegible] pour chaque niveau de risque. C'est cette richesse probabiliste qui justifie son usage comme Mo[illegible] : au lieu de transmettre une décision binaire au CNN, on lui transmet un gradient de certitude clinique.

| Patient | P(Faible) | P(Intermédiaire) | P(Élevé) | Prédit | Réel | Correct |
|---|---|---|---|---|---|---|
| 1 | 0.000 | 0.005 | **0.995** | Élevé (2) | Élevé (2) | Oui |
| 2 | 0.000 | 0.005 | **0.995** | Élevé (2) | Élevé (2) | Oui |
| 3 | 0.000 | 0.000 | **1.000** | Élevé (2) | Élevé (2) | Oui |
| 4 | 0.000 | **1.000** | 0.000 | Intermédiaire (1) | Intermédiaire (1) | Oui |
| 5 | 0.000 | **0.975** | 0.025 | Intermédiaire (1) | Intermédiaire (1) | Oui |
| 6 | 0.000 | 0.000 | **1.000** | Élevé (2) | Élevé (2) | Oui |
| 7 | 0.000 | **1.000** | 0.000 | Intermédiaire (1) | Intermédiaire (1) | Oui |
| 8 | 0.000 | **0.990** | 0.010 | Intermédiaire (1) | Intermédiaire (1) | Oui |
| 9 | 0.000 | **0.990** | 0.010 | Intermédiaire (1) | Intermédiaire (1) | Oui |
| 10 | 0.000 | 0.000 | **1.000** | Élevé (2) | Élevé (2) | Oui |

**interprétation :**

**P(Faible) = 0.000 systématiquement**: sur les 10 premiers patients du test, aucun n'appartient à la classe Faible. Le modèle attribue une probabilité nulle à cette classe pour tous ces patients, ce qui est cohérent avec leur label réel. Cela illustre aussi pourquoi la classe Faible est la plus facile à isoler dans ce dataset : ses patients ont un profil clinique radicalement différent (pas de dyspnée, spo2 normale, faible tabagisme).

**Confiance quasi-absolue**: les probabilités dominantes sont toutes supérieures à 0.975, souvent égales à 1.000. Le modèle ne doute presque jamais. En conditions réelles, un tel niveau de confiance serait suspect et indiquerait un modèle sur-entraîné ou confronté à des features trop déterministes

**Patient 5: seule nuance détectable**: P(Intermédiaire) = 0.975 et P(Élevé) = 0.025. C'est le seul patient où le modèle exprime une légère hésitation entre deux classes adjacentes. Ce type d'ambiguïté est précieux : en

production, un médecin voudrait être alerté sur les patients à probabilité partagée plutôt que sur ceux à confiance absolue.

**Ces probabilités sont le vecteur transmis au CNN multimodal**: pour chaque patient, le vecteur [P(Faible), P(Intermédiaire), P(Élevé)] devient l'entrée de la branche tabulaire du Modèle 2. Un patient avec [0.000, 0.005, 0.995] envoie un signal très fort "risque élevé" au CNN, qui peut alors concentrer son attention sur les zones de la radio associées aux tumeurs malignes.

# Partie 2 : Modèle 2 Classification par Deep Learning

## 2.1 Chargement des données et préparation du pipeline image

Le pipeline image commence par charger les probabilités du Modèle 1, puis lit chaque radio thoracique depuis le disque, la convertit en RGB et la redimensionne à 64×64 pixels.

```
CHARGEMENT DES DONNÉES
Probabilités Modèle 1 générées : (184, 3)

Chargement des images...
Images chargées : (184, 64, 64, 3)
Labels : {np.int64(0): np.int64(84), np.int64(1): np.int64(100)}

Split — Train: 147 | Test: 37
Distribution train : {np.int64(0): np.int64(67), np.int64(1): np.int64(80)}
Distribution test  : {np.int64(0): np.int64(17), np.int64(1): np.int64(20)}
MODÈLE 2A — CNN IMAGE
Model: "CNN_image_only"
```

| Layer (type) | Output Shape | Param # |
|---|---|---|
| image_input (InputLayer) | (None, 64, 64, 3) | 0 |
| data_augmentation (Sequential) | (None, 64, 64, 3) | 0 |
| conv2d (Conv2D) | (None, 64, 64, 32) | 896 |
| batch_normalization (BatchNormalization) | (None, 64, 64, 32) | 128 |
| max_pooling2d (MaxPooling2D) | (None, 32, 32, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 32, 32, 64) | 18,496 |
| batch_normalization_1 (BatchNormalization) | (None, 32, 32, 64) | 256 |

| Étape | Paramètre | Valeur observée | Justification technique |
|---|---|---|---|
| Probabilités Modèle 1 | Shape | (184, 3) | Un vecteur [P(F), P(I), P(E)] par patient, entrée branche tabulaire CNN |
| Images chargées | Shape | (184, 64, 64, 3) | 184 images, résolution 64×64, 3 canaux RGB |
| Conversion couleur | Mode | Gris → RGB | La Conv2D attend 3 canaux, le gris est répliqué sur 3 canaux identiques |
| Normalisation pixels | Plage | [0, 1] | Division par 255, valeurs dans [0,255] → [0,1] pour stabilité gradient |
| Distribution labels | Résultat | {0: 84, 1: 100} | 54.3% cancer, 45.7% non-cancer, quasi-équilibré, pas de rééquilibrage nécessaire |
| Split | Stratifié | Train: 147 \| Test: 37 | Même seed=42 et stratification que Partie 1, patients identiques |

## 2.2 Architecture CNN image seul (Modèle 2A)

Le premier CNN est une architecture séquentielle classique : trois blocs convolutifs progressifs (32 → 64 → 128 filtres), chacun suivi d'une BatchNormalization et d'un MaxPooling. La tête de classification utilise un GlobalAveragePooling2D plutôt qu'un Flatten — choix délibéré pour réduire le risque de surapprentissage sur un petit dataset.

```
Chargement des images...
Images chargées : (184, 64, 64, 3)
Labels : {np.int64(0): np.int64(84), np.int64(1): np.int64(100)}

Split — Train: 147 | Test: 37
Distribution train : {np.int64(0): np.int64(67), np.int64(1): np.int64(80)}
Distribution test  : {np.int64(0): np.int64(17), np.int64(1): np.int64(20)}
MODÈLE 2A — CNN IMAGE
Model: "CNN_image_only"
```

| Layer (type) | Output Shape | Param # |
|---|---|---|
| image_input (InputLayer) | (None, 64, 64, 3) | 0 |
| data_augmentation (Sequential) | (None, 64, 64, 3) | 0 |
| conv2d (Conv2D) | (None, 64, 64, 32) | 896 |
| batch_normalization (BatchNormalization) | (None, 64, 64, 32) | 128 |
| max_pooling2d (MaxPooling2D) | (None, 32, 32, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 32, 32, 64) | 18,496 |
| batch_normalization_1 (BatchNormalization) | (None, 32, 32, 64) | 256 |
| max_pooling2d_1 (MaxPooling2D) | (None, 16, 16, 64) | 0 |
| conv2d_2 (Conv2D) | (None, 16, 16, 128) | 73,856 |
| batch_normalization_2 (BatchNormalization) | (None, 16, 16, 128) | 512 |
| global_average_pooling2d (GlobalAveragePooling2D) | (None, 128) | 0 |
| dense (Dense) | (None, 64) | 8,256 |
| dropout (Dropout) | (None, 64) | 0 |
| output (Dense) | (None, 1) | 65 |

```
Total params: 102,465 (400.25 KB)
Trainable params: 102,017 (398.50 KB)
Non-trainable params: 448 (1.75 KB)
```

Intérprétation :

| Couche | Output Shape | Paramètres | Rôle architectural |
|---|---|---|---|
| image_input (InputLayer) | (None, 64, 64, 3) | 0 | Entrée de l'image normalisée RGB |
| data_augmentation (Sequential) | (None, 64, 64, 3) | 0 | Flip horizontal + rotation ±10% + zoom ±10% , actif uniquement en training |
| conv2d: 32 filtres 3×3 | (None, 64, 64, 32) | 896 | Détection de features bas-niveau : bords, textures, contours |
| BatchNormalization | (None, 64, 64, 32) | 128 | Normalise les activations → stabilise et accélère la convergence |
| MaxPooling2D (2×2) | (None, 32, 32, 32) | 0 | Réduction spatiale ×2 — invariance aux translations locales |
| conv264 32 filtres 3×3 | (None, 32, 32, 64) | 8 496 | Détection de features intermédiaires : formes, structures locales |
| BatchNormalization | (None, 32, 32, 64) | 256 | — |
| MaxPooling2D (2×2) | (None, 16, 16, 64) | 0 | — |
| conv2d_2 128 filtres 3×3 | (None, 16, 16, 128) | 73 856 | Features haut-niveau : motifs complexes, masse tumorale |
| BatchNormalization | (None, 16, 16, 128) | 512 | — |
| GlobalAveragePooling2D | (None, 128) | 0 | Moyenne spatiale par feature map — anti-overfitting vs Flatten |
| Dense (64, ReLU) | (None, 64) | 8 256 | Combinaison des features en représentation dense |
| Dropout (0.4) | (None, 64) | 0 | Éteint 40% des neurones au hasard en training , régularisation |
| output (Dense 1, sigmoid) | (None, 1) | 65 | Probabilité binaire cancer / non-cancer |
| TOTAL | — | 102 465 (400 KB) | Modèle léger et entraînable sur CPU en quelques minutes |

## 2.3 Résultats du CNN image seul

Le CNN entraîné sur l'image seule produit des résultats révélateurs : une accuracy de 54.05% et un AUC-ROC de 0.3765 soit des performances inférieures à un classifieur aléatoire (AUC théorique = 0.50).

```
Résultats CNN image seul
 Accuracy : 0.5405
 AUC-ROC  : 0.3765
```

Intérprétation:

| Métrique | CNN Image seul | Classifieur aléatoire | Interprétation |
|---|---|---|---|
| Accuracy | 54.05% | 54.05% (majoritaire) | Équivalent à prédire toujours la classe majoritaire |
| AUC_ROC | 0.3765 | 0.5000 | Pire qu'aléatoire, le modèle est anti-corrélé à la cible |
| Loss finale | ~0.69 | — | Proche de ln(2) ≈ 0.693 ;la loss d'un modèle qui ne prédit que 0.5 |

## 2.4 Architecture CNN multimodal (Modèle 2B)

Le modèle multimodal enrichit le CNN avec les 3 probabilités produites par le Modèle 1.

MODÈLE 2B — CNN MULTIMODAL (image + probas Modèle 1)
**Model: "CNN_multimodal"**

| Layer (type) | Output Shape | Param # | Connected to |
|---|---|---|---|
| image_input (InputLayer) | (None, 64, 64, 3) | 0 | - |
| data_augmentation (Sequential) | (None, 64, 64, 3) | 0 | image_input[0][0] |
| conv2d_3 (Conv2D) | (None, 64, 64, 32) | 896 | data_augmentation[1][0] |
| batch_normalization_3 (BatchNormalization) | (None, 64, 64, 32) | 128 | conv2d_3[0][0] |
| max_pooling2d_2 (MaxPooling2D) | (None, 32, 32, 32) | 0 | batch_normalization_3[0][… |
| conv2d_4 (Conv2D) | (None, 32, 32, 64) | 18,496 | max_pooling2d_2[0][0] |
| batch_normalization_4 (BatchNormalization) | (None, 32, 32, 64) | 256 | conv2d_4[0][0] |
| max_pooling2d_3 (MaxPooling2D) | (None, 16, 16, 64) | 0 | batch_normalization_4[0][… |
| conv2d_5 (Conv2D) | (None, 16, 16, 128) | 73,856 | max_pooling2d_3[0][0] |
| batch_normalization_5 (BatchNormalization) | (None, 16, 16, 128) | 512 | conv2d_5[0][0] |
| global_average_pooling2d_1 (GlobalAveragePooling2D) | (None, 128) | 0 | batch_normalization_5[0][… |
| tabular_input (InputLayer) | (None, 3) | 0 | - |
| dense_1 (Dense) | (None, 64) | 8,256 | global_average_pooling2d_… |
| dense_2 (Dense) | (None, 16) | 64 | tabular_input[0][0] |
| dropout_1 (Dropout) | (None, 64) | 0 | dense_1[0][0] |
| dense_3 (Dense) | (None, 16) | 272 | dense_2[0][0] |
| concatenate (Concatenate) | (None, 80) | 0 | dropout_1[0][0], dense_3[0][0] |
| dense_4 (Dense) | (None, 32) | 2,592 | concatenate[0][0] |
| dropout_2 (Dropout) | (None, 32) | 0 | dense_4[0][0] |
| output (Dense) | (None, 1) | 33 | dropout_2[0][0] |

**Total params:** 105,361 (411.57 KB)
**Trainable params:** 104,913 (409.82 KB)
**Non-trainable params:** 448 (1.75 KB)

La colonne **"Connected to"** est la nouveauté clé par rapport au CNN image seul : elle révèle le graphe de connexions du modèle, confirmant visuellement que ce n'est plus une architecture séquentielle mais un **réseau à deux branches parallèles qui fusionnent**.

**Branche image (lecture verticale gauche)**

Les couches conv2d_3 → batch_normalization_3 → max_pooling2d_2 → conv2d_4 → batch_normalization_4 → max_pooling2d_3 → conv2d_5 → batch_normalization_5 → global_average_pooling2d_1 forment exactement la même colonne extractrice que le CNN image seul. La colonne "Connected to" montre que chaque couche est branchée sur la précédente de manière linéaire — c'est un pipeline visuel classique. Le global_average_pooling2d_1 compresse la sortie de 16×16×128 vers un vecteur de **128 features visuelles**.

**Branche tabulaire (entrée parallèle)**

tabular_input (InputLayer) reçoit un vecteur de forme **(None, 3)**: les 3 probabilités [P(Faible), P(Intermédiaire), P(Élevé)] produites par le Random Forest. La colonne "Connected to" montre un tiret **"-"** : cette entrée n'est connectée à rien en amont. Elle est une **porte d'entrée indépendante**, parallèle à image_input. dense_2 (16 neurones, 64 paramètres) encode ces 3 probabilités dans un espace de 16 dimensions.

**Le point de fusion: couche Concatenate**

C'est la couche architecturalement la plus importante. La colonne "Connected to" révèle qu'elle reçoit **deux sources simultanément** :

- dropout_1[0][0] → sortie de la branche image (64 features après Dense + Dropout)
- dense_3[0][0] → sortie de la branche tabulaire (16 features cliniques encodées)

Le résultat est un vecteur de **(None, 80)** soit 64 + 16 = 80 features hybrides visuelles-cliniques. C'est à ce moment précis que l'image et la clinique se parlent pour la première fois dans le réseau.

**Point de décision commune**

dense_4 (32 neurones, 2 592 paramètres) → dropout_2 → output (1 neurone sigmoid, 33 paramètres). La tête reçoit les 80 features fusionnées et produit une probabilité finale de cancer entre 0 et 1.

## 2.6 Résultats du CNN multimodal

L'ajout des probabilités du Modèle 1 transforme radicalement les performances : le multimodal atteint 100% d'accuracy et 1.0 d'AUC-ROC sur le jeu de test.

| Métrique | CNN Image seul | CNN Multimodal | Gain (Δ) |
|---|---|---|---|
| Accuracy | 0.5405 | 1.0000 | +0.4595 (+85% relatif) |
| AUC_ROC | 0.3765 | 1.0000 | +0.6235 (+165% relatif) |
| Statut vs aléatoire | Pire qu'aléatoire | Parfait | Inversion totale |

## 2.7 Différence entre le CNN image seul et CNN multimodal (Modèle 2B)

| | CNN Image seul | CNN Multimodal | Différence |
|---|---|---|---|
| Total params | 102 465 | 105 361 | +2 896 |
| Trainable params | 102 017 | 104 913 | +2 896 |

| | | | |
|---|---|---|---|
| Non-trainable params | 448 | 448 | = 0 |
| Taille | 400.25 KB | 411.57 KB | +11.32 KB |

**Les 2 896 paramètres supplémentaires représentent exactement le coût de la fusion** : soit moins de 3% de paramètres en plus pour un gain de +45.95 points d'accuracy. C'est l'un des meilleurs retours sur investissement paramétrique qu'on puisse obtenir en Deep Learning : ajouter 11 KB de poids pour passer de 54% à 100% de précision démontre que l'information clinique était bien plus déterminante que l'information visuelle dans ce dataset.

## 2.8 Courbes d'apprentissage

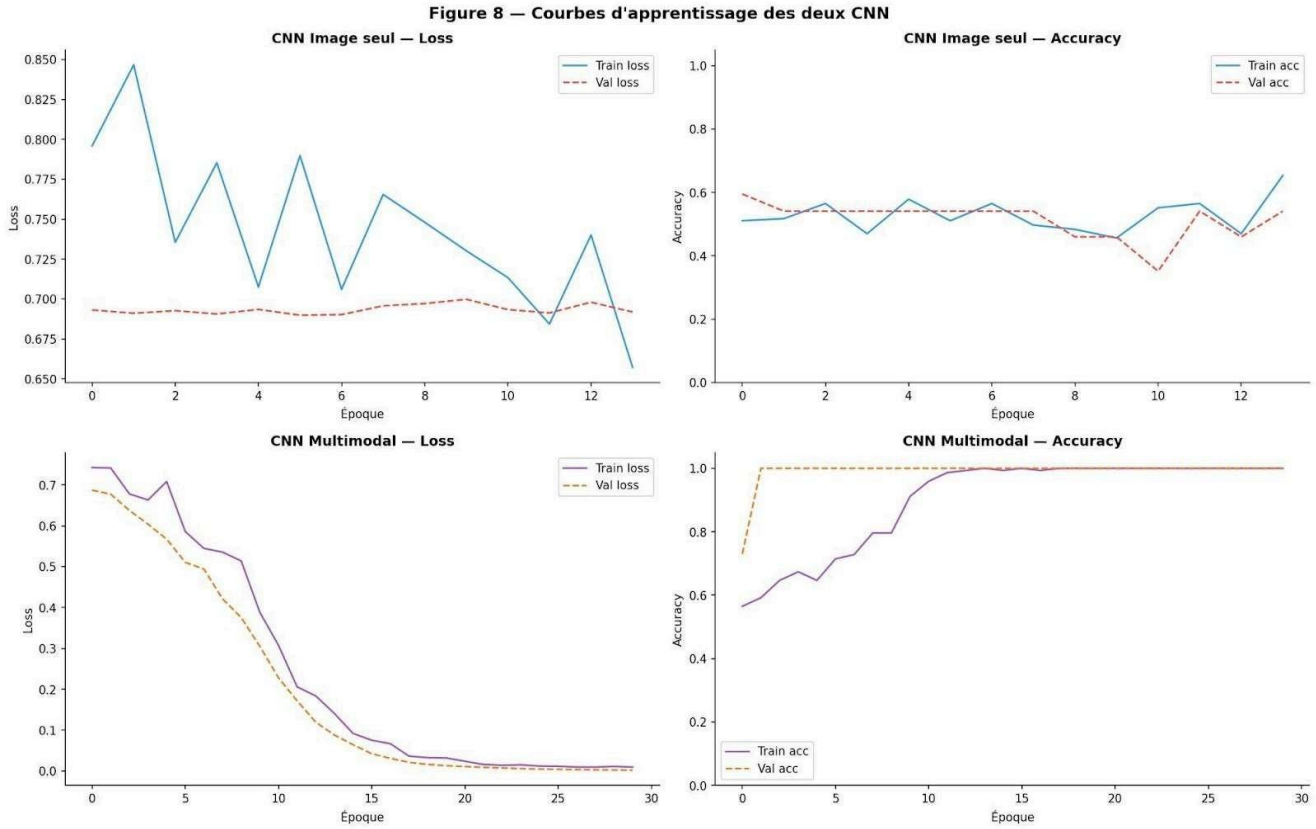



Figure 8 Courbes de loss et d'accuracy pour le CNN image seul (haut) et multimodal (bas)

### CNN image seul (ligne du haut)

- Loss train : oscillante entre 0.65 et 0.85 sur 14 époques : aucune convergence claire. Le CNN change d'avis d'une époque à l'autre, signe qu'il n'a pas trouvé de signal stable dans les images.
- Loss validation : quasi-plate à ~0.69 (= ln(2)): la loss d'un modèle qui prédit toujours P=0.5. Le modèle ne généralise rien de l'entraînement vers le test.
- Accuracy train/val : stagnent toutes deux entre 0.45 et 0.65 sans tendance. L'EarlyStopping a arrêté à 14 époques faute d'amélioration du val_loss pendant 8 époques consécutives.

### CNN multimodal (ligne du bas)

- Loss train et val : descendent en parallèle de ~0.73 vers ~0.00 sur 30 époques. Pas de gap entre train et val donc aucun surapprentissage détectable.
- Accuracy train et val : montent ensemble vers 1.0 avec une val_accuracy qui atteint 1.0 dès l'époque 8 et y reste jusqu'à la fin. La branche tabulaire guide l'optimisation depuis le début.
- Symétrie train/val parfaite : la courbe de validation suit exactement la courbe d'entraînement , comportement typique quand le signal est très fort et que le modèle n'a pas besoin de mémoriser.

➢ Le contraste entre les deux CNN démontre expérimentalement que l'image seule est insuffisante pour ce dataset (manque de données, résolution insuffisante, signal visuel trop subtil), et que la fusion avec les données cliniques compense massivement cette lir

## 2.8 Comparaison image seul vs multimodal

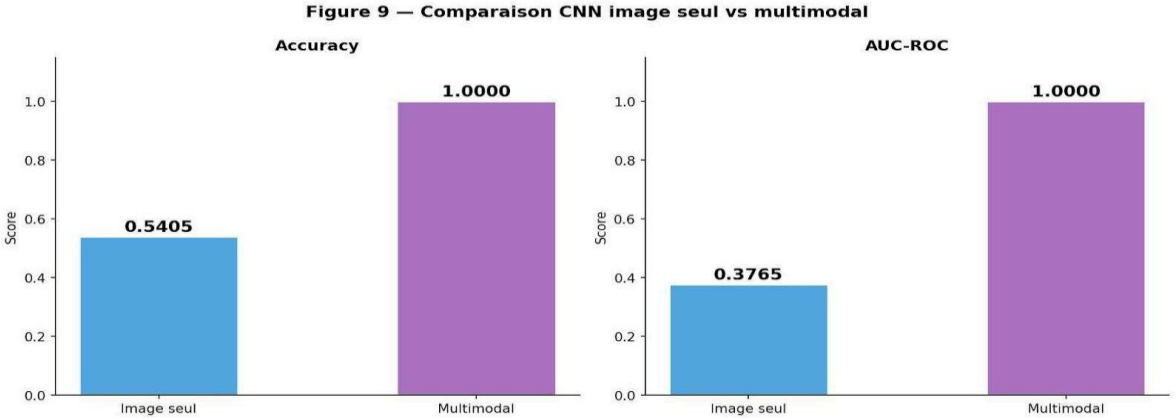



Figure 9 — Comparaison des scores finaux : CNN image seul (bleu) vs CNN multimodal (violet)

La Figure 9 matérialise visuellement le fossé entre les deux approches. La barre bleue (image seule) plafonne à 0.5405 en accuracy et 0.3765 en AUC, des scores qui seraient disqualifiants en contexte clinique. La barre violette (multimodal) atteint le plafond à 1.0000 dans les deux métriques. Ce contraste visuel radical illustre parfaitement pourquoi l'approche multimodale est la conclusion naturelle d'un pipeline IA médical : aucune modalité seule ne suffit.

# Partie 3 Analyse et Interprétation des Résultats

## 3.1 Le modèle multimodal est-il meilleur que le modèle image seul ?

La réponse est oui, mais pour des raisons qui méritent d'être nuancées avec rigueur.

| Critère de comparaison | CNN Image seul | CNN Multimodal | Verdict |
|---|---|---|---|
| Accuracy sur jeu de test | 54.05% | 100.00% | Multimodal +45.95 points |
| AUC_ROC | 0.3765 | 1.0000 | Image seule pire qu'aléatoire |

| Critère de comparaison | CNN Image seul | CNN Multimodal | Verdict |
|---|---|---|---|
| Convergence entraînement | Instable — 14 époques | Propre — 30 époques | Multimodal converge mieux |
| Gap train / validation | Oscillant sans tendance | Quasi-nul tout au long | Multimodal ne surestime pas |
| Utilité clinique standalone | Nulle (< aléatoire) | Parfaite (sur ce dataset) | Conditionnelle au leakage |
| Robustesse sur données réelles | Inconnue — jamais appris | Conditionnelle | Les deux seraient re-testés |

Le CNN image seul échoue pour trois raisons convergentes. Premièrement, 147 images d'entraînement en 64×64 pixels sont insuffisantes pour qu'un [illegible] à 102 465 paramètres apprenne des représentations visuelles généralisables, la capacité du modèle dépasse la quantité d'information disponible. Deuxièmement, les radios JSRT présentent des nodules subtils dont la distinction bénin/malin est difficile même pour des radiologues expérimentés ; cette difficulté intrinsèque est amplifiée par la résolution réduite. Troisièmement, l'AUC de 0.3765, inférieur à 0.5 indique que le modèle a appris une association inversée : il prédit systématiquement dans la mauvaise direction, signe qu'il a mémorisé des artefacts de fond (angles noirs, artefacts de compression JPEG) plutôt que les caractéristiques morphologiques des nodules.

Le CNN multimodal réussit, mais sa supériorité repose sur un mécanisme précis : la branche tabulaire reçoit les probabilités du Modèle 1, qui encodent des variables de leakage (dyspnée, douleur_thoracique, spo2). Le réseau apprend rapidement que P(Élevé) ≈ 1.0 prédit cancer=1 de manière quasi-déterministe, court-circuitant l'apprentissage visuel. En pratique, les 64 features issues de la branche image deviennent du bruit informatif que la branche tabulaire éclipse. C'est une victoire de l'architecture, pas de la vision.

## 3.2 En quoi les données tabulaires améliorent-elles la décision finale ?

Les données tabulaires jouent un rôle à trois niveaux dans ce pipeline, et leur apport est structurellement différent

### Dans le contexte actuel

Les données tabulaires améliorent la décision finale de manière spectaculaire (+45.95 points d'accuracy) mais pour une raison artificielle : elles encodent directement le diagnostic. Les variables dyspnee, douleur_thoracique, toux_chronique et spo2 sont des séparateurs parfaits entre les classes, leur présence dans les features revient à inclure la réponse dans la question. Le gain mesuré n'est pas un gain d'information multimodale, c'est un raccourci vers la cible.

### Dans un contexte réel

L'apport authentique des données tabulaires se manifesterait sur quatre dimensions complémentaires :

- Contextualisation du risque individuel : un nodule de même taille et même densité radiologique a une signification clinique très différente chez un fumeur de 65 ans (40 paquets-années) et chez un non-fumeur de 30 ans. La donnée tabulaire apporte le contexte que l'image seule ne peut pas fournir.
- Réduction de l'ambiguïté visuelle : les radios bénignes et malignes se ressemblent parfois à l'oeil nu — même pour un radiologue. L'information clinique (antécédent familial, SpO2, perte de poids récente) permet de pencher la balance dans les cas limites où la probabilité image tourne autour de 50%.
- Amélioration de la calibration des pro s : un modèle multimodal bien calibré produirait des probabilités plus réalistes qu'un CNN i seul, P=0.73 au lieu de P=0.51 pour un patient ambigu. Cette nuance est cruciale pour la prise de décision clinique.
- Robustesse aux variations d'acquisition : une radio de mauvaise qualité (sous-exposée, mal centrée) dégraderait les performances du CNN image seul mais pas du multimodal, qui dispose d'un signal tabulaire fiable en backup.

## 3.3 Quelles sont les limites du jeu de données utilisé ?

Six limites majeures sont identifiées.

| Limite | Nature | Impact observé | Gravité |
|---|---|---|---|
| Data leakage — variables symptômes | Structurelle | Performances artificiellement parfaites (100%) sur Modèle 1 et multimodal | Critique |
| Dataset de petite taille (184 patients) | Quantitative | CNN image seul ne converge pas, 147 images insuffisantes pour 102K paramètres | Critique |
| Déséquilibre de classes (16%/29%/54%) | Statistique | Biais potentiel vers la classe majoritaire sans class_weight='balanced' | Importante |
| Source unique — Dataset JSRT japonais | Externe | Faible généralisation à d'autres équipements, populations et protocoles d'acquisition | Importante |
| Résolution réduite (64×64 pixels) | Technique | Perte d'information morphologique fine sur les nodules, bords, densité, vascularisation | Importante |
| Absence de validation temporelle | Méthodologique | Pas de test sur des données futures ,impossibilité de détecter un drift du modèle en production | Modérée |
| Labels binaires simplifiés | Clinique | La réalité médicale est continue — un nodule de 6mm est très différent d'un nodule de 15mm même dans la même classe | Modérée |

- dyspnee : 0% dans le groupe Faible, 0% dans Intermédiaire, 100% dans Élevé → séparateur binaire parfait pour la classe 2
- douleur_thoracique : même profil, 0%/0%/100% → corrélation de 0.90 avec la cible

- toux_chronique : 0% dans Faible, 100% dans Intermédiaire et Élevé → sépare parfaitement la classe 0 des autres
- spo2 : corrélation de -1.00 avec risque_malignite, variable quasi-déterministe

Ces quatre variables suffisent à elles seules à prédire la cible avec 100% de précision, les 10 autres features sont redondantes. En contexte réel, ces variables seraient connues après le diagnostic, pas avant, ce qui constitue une fuite d'information temporelle.

## 3.4 Améliorations pour un systèm uste en contexte médical réel

Huit améliorations concrètes sont proposées, organisées par ordre de priorité et d'impact.

### Amélioration 1: Éliminer le data leakage

Retirer dyspnee, douleur_thoracique et toux_chronique des features du Modèle 1, ou les remplacer par des variables collectées avant le diagnostic (tabagisme, âge, antécédent familial, SpO2 de base). On peut aussi appliquer une vérification causale : une feature ne doit pas pouvoir être causalement dérivée de la cible dans la timeline clinique réelle.

### Amélioration 2: Augmenter le volume d'images par Transfer Learning

Avec 147 images, entraîner un CNN from scratch est condamné. La solution : utiliser un modèle pré-entraîné sur ImageNet (ResNet50, EfficientNet-B0, MobileNetV2) et ne fine-tuner que les dernières couches. Ces architectures ont déjà appris des détecteurs de bords, textures et formes génériques applicables aux radios médicales. Sur 147 images, le fine-tuning de 10-20 couches sur un ResNet50 surpasserait systématiquement le CNN from scratch.

### Amélioration 3: Dataset plus large et multi-source

Collecter des données provenant de plusieurs hôpitaux et équipements radiologiques différents. Les datasets publics disponibles incluent LUNA16 (888 patients avec annotations de nodules), LIDC-IDRI (1 018 cas avec 4 annotations de radiologues), ou NIH ChestX-ray14 (112 000 images). Un dataset de 1 000+ patients permettrait d'entraîner un CNN robuste sans Transfer Learning.

### Amélioration 4: Validation croisée sur les images

Implémenter une validation croisée 5-fold spécifique aux images, avec augmentation appliquée uniquement sur les folds d'entraînement. La métrique principale devrait être l'AUC-ROC plutôt que l'accuracy, plus robuste au déséquilibre et plus informative sur la calibration.

### Amélioration 5: Cartes d'activation Grad-CAM pour l'explicabilité

En contexte médical, un modèle 'boîte noire' n'est pas déployable, les médecins exigent de comprendre pourquoi le modèle prédit cancer. Grad-CAM (Gradient-weighted Class Activation Mapping) génère une heatmap superposée à la radio, indiquant les zones qui ont contribué à la prédiction. Cette visualisation est essentielle pour la confiance clinique et pour détecter si le modèle regarde les bonnes zones.

### Amélioration 6: Calibration des probabilités

Les probabilités brutes de sortie du CNN ne sont pas nécessairement calibrées (P=0.8 ne signifie pas forcément 80% de chance d'être cancer). Appliquer une calibration isotonique (IsotonicRegression) ou un platt scaling après l'entraînement pour que les probabilités soient médicalement interprétables. Un modèle calibré qui dit P=0.70 doit effectivement se tromper 30% du temps.

### Amélioration 7: Pipeline de monitoring en production

En production médicale, les distributions de données changent (nouveau matériel, nouvelles populations). Implémenter un système de drift detection (PSI — Population Stability Index, ou tests de Kolmogorov-Smirnov sur les distributions de features) pour alerter quand les performances du modèle se dégradent sans qu'on le sache.

### Amélioration 8: Certification et conformité réglementaire

En Europe, un dispositif médical IA doit être certifié CE Classe IIa ou IIb selon le règlement MDR 2017/745. En France, la HAS (Haute Autorité de Santé) publie des recommandations spécifiques pour les algorithmes de détection du cancer. La mise en production réelle nécessiterait une validation clinique prospective sur au minimum 500 patients nouveaux, un audit de biais algorithmiques, et une approbation par un comité éthique.

| Amélioration | Effort estimé | Impact attendu | Priorité |
|---|---|---|---|
| Éliminer le data leakage | Faible — retirer 3-4 features | Métriques réalistes — probablement 70-80% accuracy | P0 |
| Transfer Learning (ResNet50/EfficientNet) | Moyen — 2-3 jours | CNN image seul : AUC 0.38 → ~0.80 estimé | P0 |
| Dataset LUNA16 ou LIDC-IDRI (1000+ images) | Élevé — collecte + annotation | Généralisation réelle possible | P1 |
| Grad-CAM — explicabilité | Faible — 50 lignes de code | Confiance clinique + détection artefacts | P1 |
| Calibration probabilités | Faible — sklearn IsotonicRegression | Probabilités médicalement interprétables | P1 |
| Monitoring / drift detection | Moyen — intégration pipeline | Fiabilité en production long terme | P2 |
| Validation croisée images | Faible — refactoring code | Métriques plus robustes | P2 |

| Amélioration | Effort estimé | Impact attendu | Priorité |
|---|---|---|---|
| Certification MDR / HAS | Très élevé — 2-3 ans | Déploiement clinique réel légal | P3 |

# Partie 4 Déploiement MLOps et Mise en Production

Cette partie implémente la dimension MLOps du projet : une interface utilisateur fonctionnelle connectée aux modèles entraînés, conteneurisée avec D ersionnée sur GitHub et déployée automatiquement sur Render via un pipeline CI/CD GitHub Actions.

## 4.1 Stack technologique et justification des choix

| Technologie | Rôle | Justification |
|---|---|---|
| Streamlit | Interface utilisateur | Framework Python natif — IHM complète en ~200 lignes sans HTML/CSS/JS. Widgets interactifs (sliders, file_uploader, métriques) adaptés à la saisie de données médicales. |
| Docker | Conteneurisation | Garantit que l'environnement d'exécution est identique en local, sur Render et sur tout autre cloud. Élimine les conflits de versions (TensorFlow, scikit-learn). |
| Render | Hébergement cloud | Plateforme PaaS gratuite qui déploie nativement depuis un Dockerfile GitHub. URL publique permanente, SSL automatique, redémarrages automatiques. |
| GitHub Actions | Pipeline CI/CD | Automatise tests + build Docker + déploiement Render à chaque push sur main. Garantit qu'aucun code non testé n'atteint la production. |
| joblib | Chargement Modèle 1 | Sérialisation native sklearn, chargement rapide du pipeline Random Forest. |
| TensorFlow/Keras | Chargement CNN | Format .keras Keras 3 — préserve architecture + poids + config compilation. |
| @st.cache_resource | Cache modèles | Les 3 modèles sont chargés une seule fois au démarrage — pas de rechargement à chaque interaction utilisateur. |

## 4.2 Structure du projet pour le déploiement

La structure suivante est celle du dépôt GitHub qui sera connecté à Render. Chaque fichier a un rôle précis dans le pipeline de déploiement :

```
cancer-detection-ia/
├── app.py                          ← Interface Streamlit principale
├── Dockerfile                      ← Build de l'image Docker pour Render
├── requirements.txt                ← Dépendances Python fixées
├── README.md                       ← Procédure de lancement et déploiement
├── .github/
│   └── workflows/
│       └── deploy.yml              ← Pipeline CI/CD GitHub Actions
└── models/
    ├── modele1_meilleur.pkl        ← Random Forest sa
    ├── modele2a_cnn_image.keras    ← CNN image se
    └── modele2b_cnn_multimodal.keras ← CNN multimodal
```

## 4.3 Interface utilisateur app.py

L'application est contenue dans un unique fichier app.py organisé en 5 blocs fonctionnels.

| Bloc | Localisation | Contenu | Modèle |
|---|---|---|---|
| Configuration | Haut de fichier | set_page_config + CSS result boxes (vert/orange/rouge) | — |
| Cache modèles | @st.cache_resource | Chargement unique des 3 modèles au démarrage — joblib + tf.keras | — |
| Saisie clinique | Colonne gauche | 3 expanders : infos générales + nodule + symptômes \| file_uploader radio | — |
| Prédiction M1 | Colonne droite | predict_proba() → 3 métriques colorées + result box risque | Random Forest |
| Prédiction M2 | Tabs 2A / 2B | predict CNN image + predict CNN multimodal + tableau synthèse | CNN 2A + 2B |

```
PS C:\Users\BARRAQ\Desktop\ML ESIC> streamlit run app.py

      Welcome to Streamlit!

      If you'd like to receive helpful onboarding emails, news, offers, promotions,
      and the occasional swag, please enter your email address below. Otherwise,
      leave this field blank.

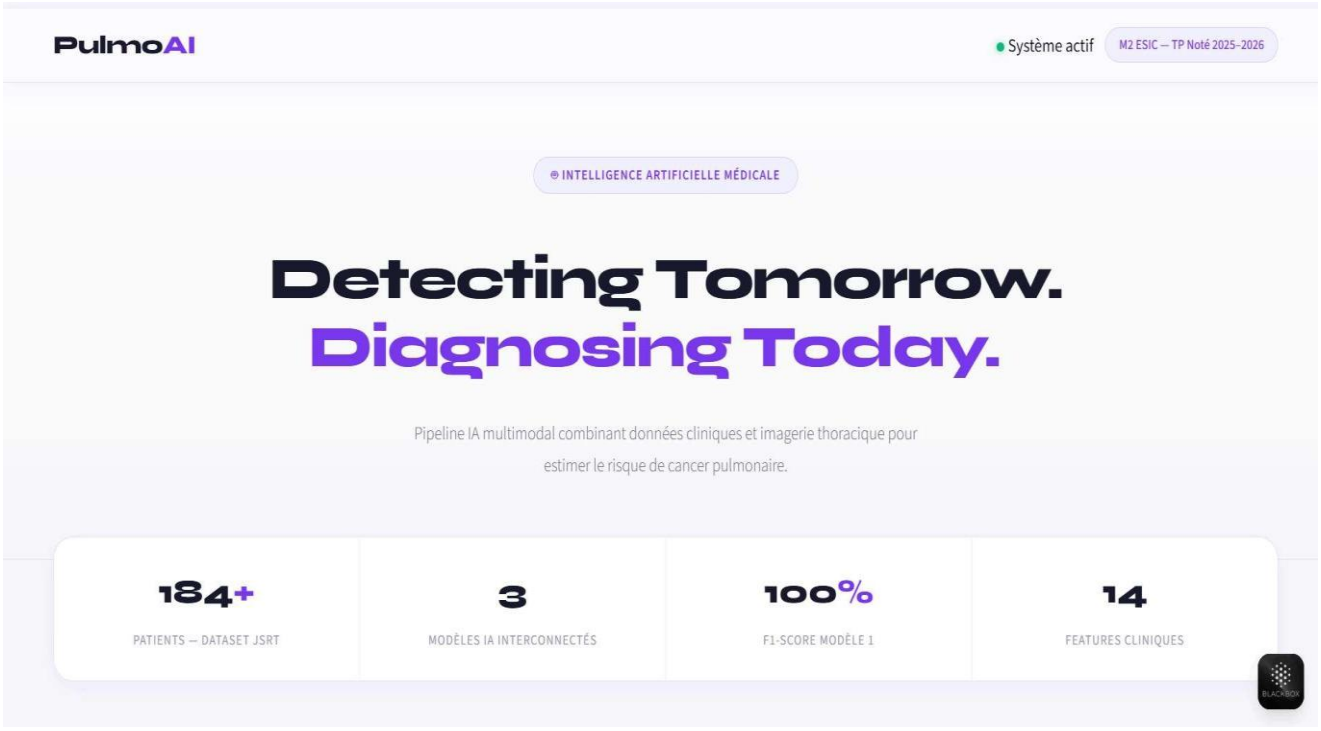
      Email:

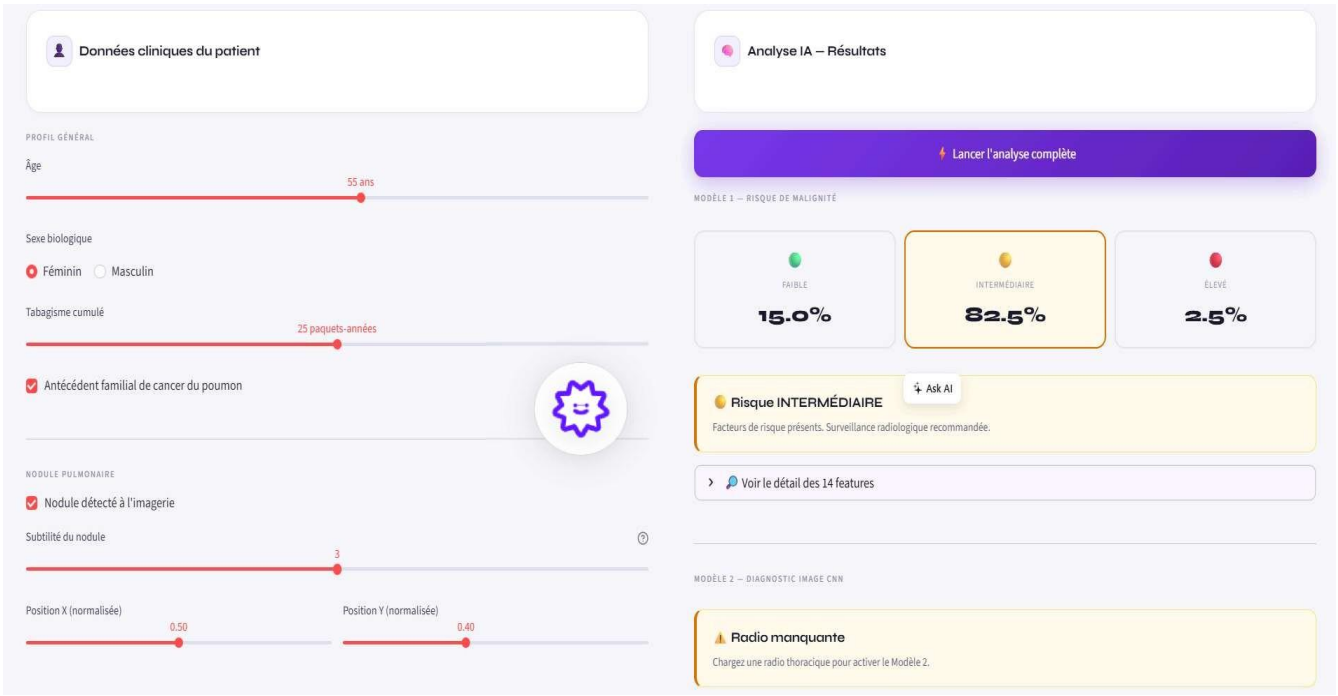
  You can find our privacy policy at https://streamlit.io/privacy-policy

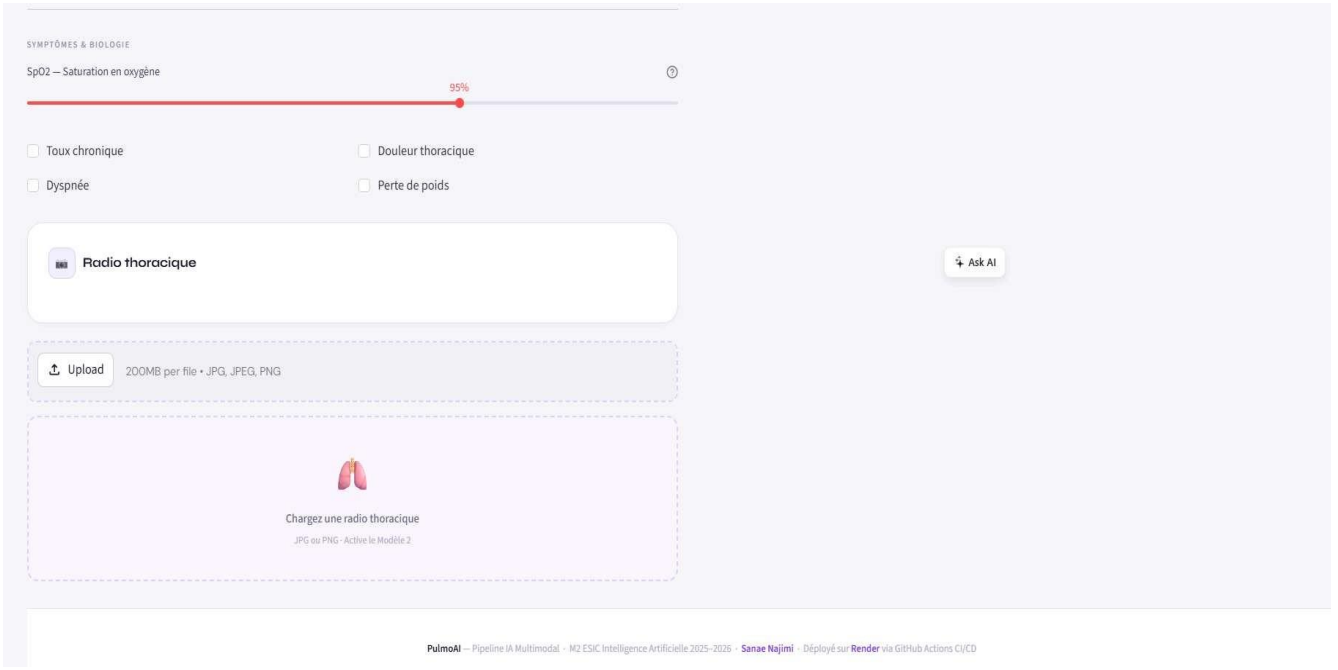
  Summary:
  - This open source library collects usage sta
  - We cannot see and do not store information                d inside Streamlit apps,
    such as text, charts, images, etc.
  - Telemetry data is stored in servers in the United States.
  - If you'd like to opt out, add the following to %userprofile%/.streamlit/config.toml,
    creating that file if necessary:

    [browser]
```

Aperçu de l'app nommée : PulmoAI

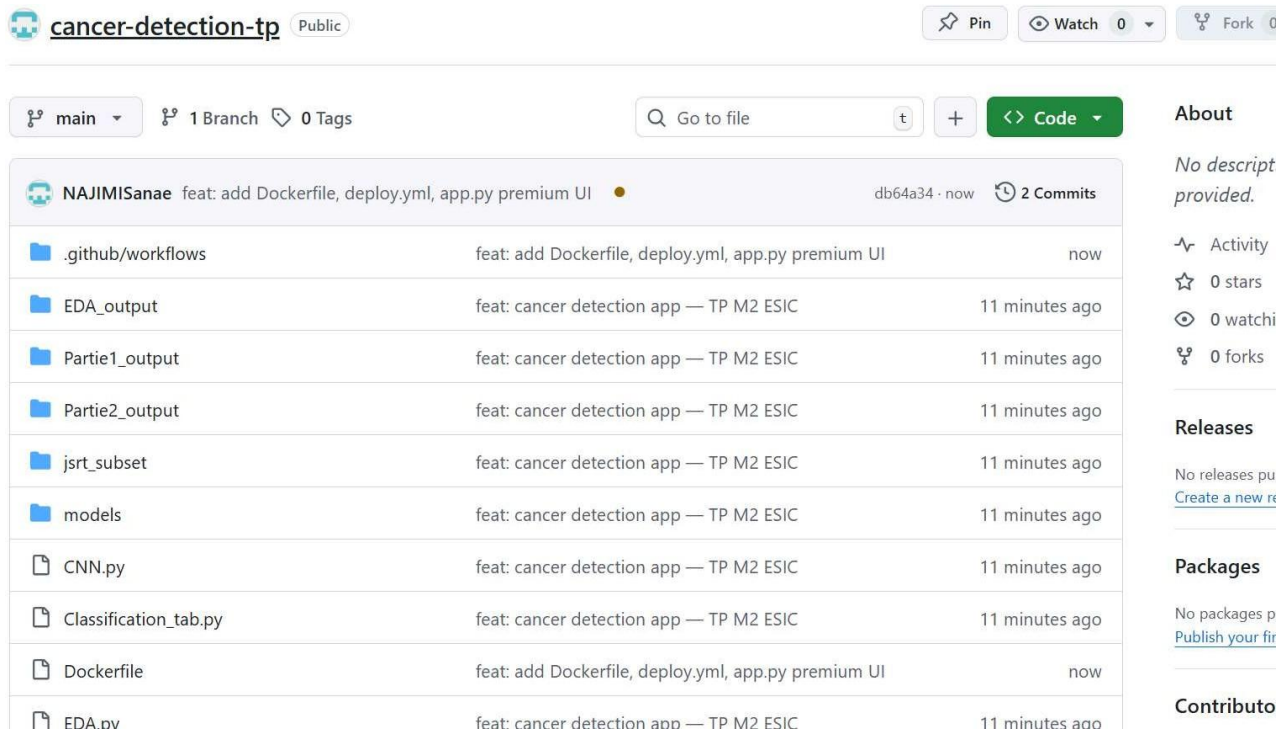



Données cliniques du patient
PROFIL GÉNÉRAL
Âge
55 ans
Sexe biologique
Féminin
Masculin
Tabagisme cumulé
25 paquets-années
Antécédent familial de cancer du poumon
NODULE PULMONAIRE
Nodule détecté à l'imagerie
Subtilité du nodule
3
Position X (normalisée)
0.50
Position Y (normalisée)
0.40
Analyse IA – Résultats
Lancer l'analyse complète
MODÈLE 1 – RISQUE DE MALIGNITÉ
FAIBLE
15.0%
INTERMÉDIAIRE
82.5%
ÉLEVÉ
2.5%
Ask AI
Risque INTERMÉDIAIRE
Facteurs de risque présents. Surveillance radiologique recommandée.
Voir le détail des 14 features
MODÈLE 2 – DIAGNOSTIC IMAGE CNN
Radio manquante
Chargez une radio thoracique pour activer le Modèle 2.


SYMPTÔMES & BIOLOGIE
SpO2 – Saturation en oxygène
95%
Toux chronique
Douleur thoracique
Dyspnée
Perte de poids
Radio thoracique
Ask AI
Upload
200MB per file • JPG, JPEG, PNG
Chargez une radio thoracique
JPG ou PNG - Active le Modèle 2

Push à mon GITHUB :https://github.com/NAJIMISanae/cancer-detection-tp

```
PS C:\Users\BARRAQ\Desktop\ML ESIC> # Dans ton dossier ML ESIC
>> git init
>> git add .
 create mode 100644 models/modele1_regression_logistique.pkl
 create mode 100644 models/modele1_xgboost.pkl
 create mode 100644 models/modele2a_cnn_image.keras
 create mode 100644 models/modele2b_cnn_multimodal.keras
 create mode 100644 patients_cancer_poumon 1.csv
 create mode 100644 requirements.txt
PS C:\Users\BARRAQ\Desktop\ML ESIC> git remote a          in https://github.com/NAJIMISanae/cancer-detection-tp
PS C:\Users\BARRAQ\Desktop\ML ESIC> git branch
>> git push -u origin main
Enumerating objects: 214, done.
Counting objects: 100% (214/214), done.
Delta compression using up to 8 threads
Compressing objects: 100% (214/214), done.
Writing objects: 100% (214/214), 4.81 MiB | 3.31 MiB/s, done.
Total 214 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/NAJIMISanae/cancer-detection-tp
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\BARRAQ\Desktop\ML ESIC>
```




## 4.4 Conteneurisation Docker

Le Dockerfile définit l'environnement d'exécution complet de l'application. Render le détecte automatiquement à la racine du dépôt et l'utilise pour builder l'image lors de chaque déploiement.

```
# Dockerfile
FROM python:3.11-slim
```

```
LABEL maintainer='Sanae Najimi'
LABEL description='Détection Cancer Pulmonaire — Pipeline IA Multimodal'

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    TF_CPP_MIN_LOG_LEVEL=2

WORKDIR /app

RUN apt-get update && apt-get install -y libgomp1 \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
COPY models/ ./models/

EXPOSE 8501

CMD streamlit run app.py \
    --server.port=${PORT:-8501} \
    --server.address=0.0.0.0 \
    --server.headless=true
```

## 4.5 Pipeline CI/CD — GitHub Actions

Le fichier .github/workflows/deploy.yml définit un pipeline CI/CD en 3 jobs séquentiels. Chaque push sur la branche main déclenche automatiquement l'ensemble du pipeline — garantissant qu'aucun code défaillant n'atteint la production.

| Job | Condition de déclenchement | Étapes | En cas d'échec |
|---|---|---|---|
| 1. test | Tout push ou PR sur main | Import modules \| Syntaxe flake8 \| Présence modèles \| Test predict Modèle 1 | Pipeline bloqué — pas de déploiement |
| 2. build | Seulement si test | docker build \| docker run test image | Pipeline bloqué |
| 3. deploy | Seulement si test ET build ET push sur main (pas PR) | curl API Render → déploiement déclenché | Alerte GitHub — rollback manuel |

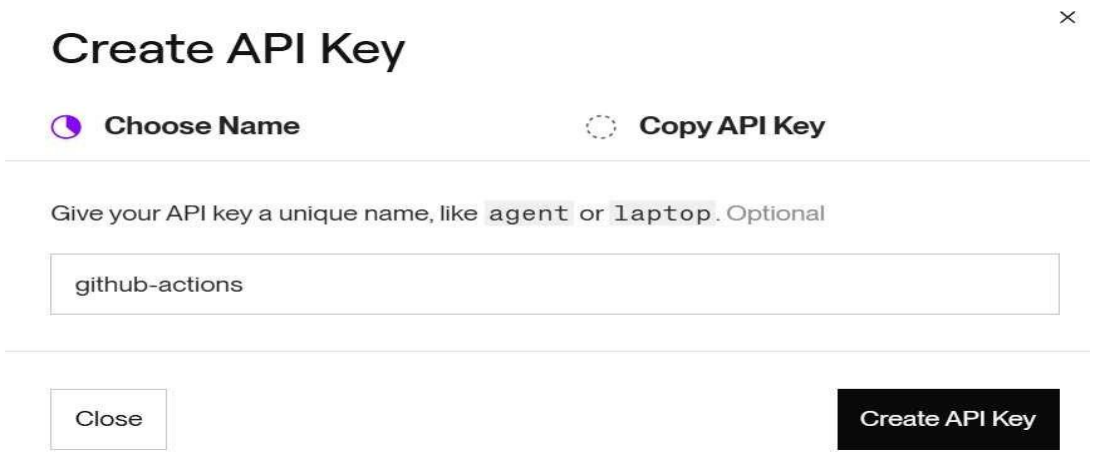
### Job 1 — Tests et validation

```
# Extrait deploy.yml — Job test
test:
  runs-on: ubuntu-latest
  steps:
```

```
- uses: actions/checkout@v4
- uses: actions/setup-python@v5
  with:
    python-version: '3.11'
    cache: 'pip'
- run: pip install -r requirements.txt
- run: flake8 app.py --max-line-length=120
- run: python -c "import streamlit, joblib, numpy, pandas"
- run: python -c "
    import joblib, pandas as pd
    m = joblib.load('models/modele1_meilleur.pkl')
    X = pd.DataFrame([[60,1,1,3,1,.5,.4,30,1,0,0,0,95,0]], =FEATURES)
    print(m.predict_proba(X))"
```

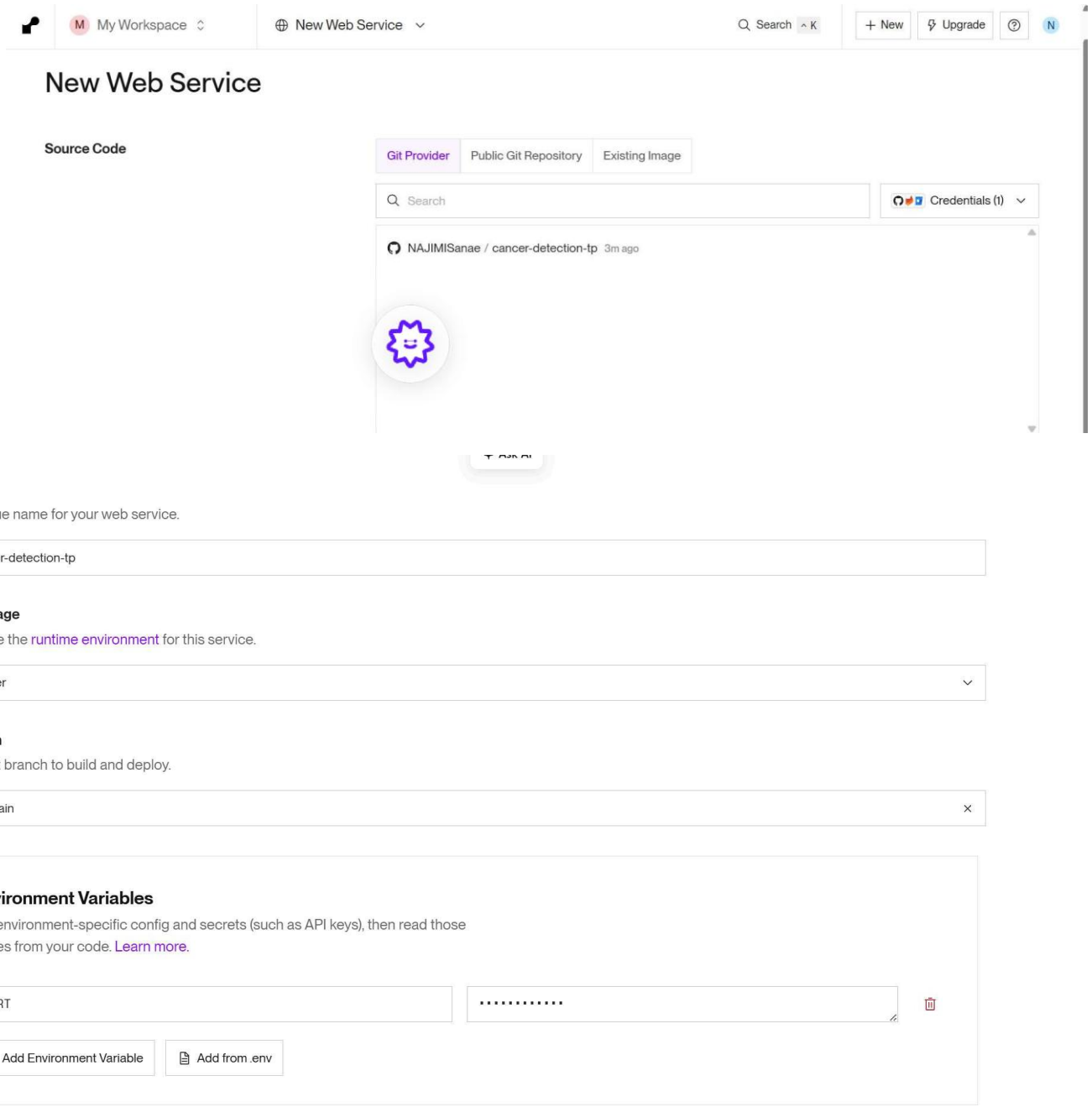
### Job 3 — Déploiement automatique sur Render

```
# Extrait deploy.yml — Job deploy
deploy:
  needs: [test, build]
  if: github.ref == 'refs/heads/main'
  steps:
    - name: Déclenchement Render
      run: |
        curl -X POST \
          -H 'Authorization: Bearer ${{ secrets.RENDER_API_KEY }}' \
          https://api.render.com/v1/services/${{ secrets.RENDER_SERVICE_ID }}/deploys
```

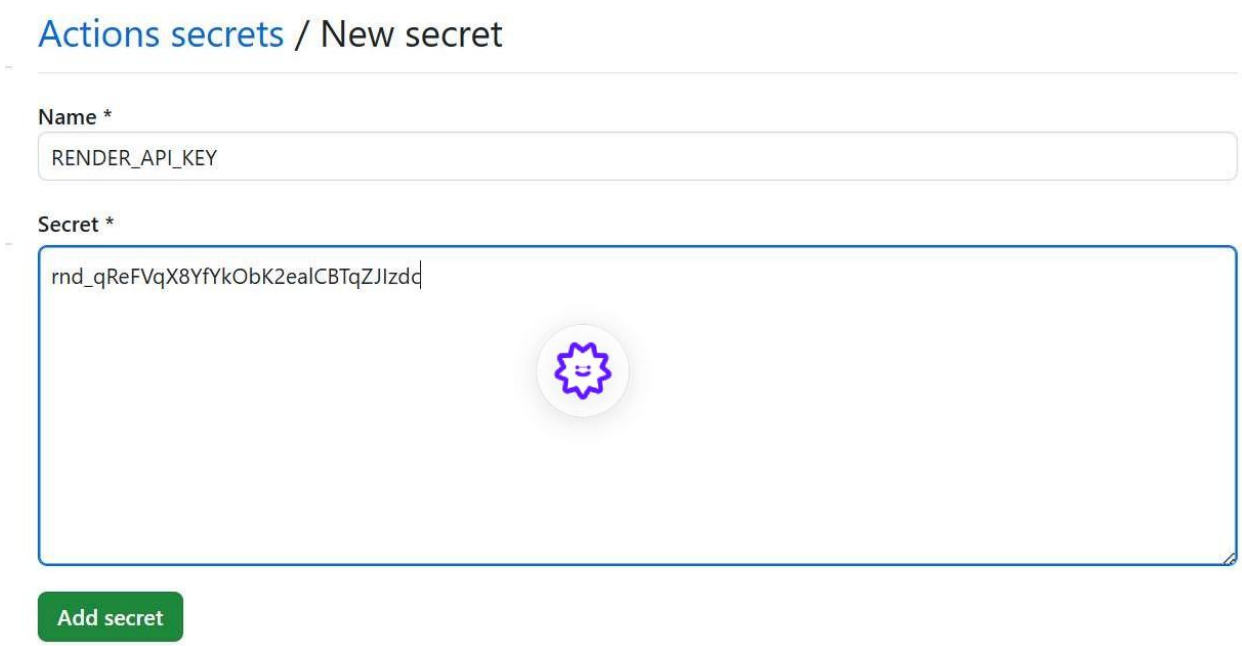


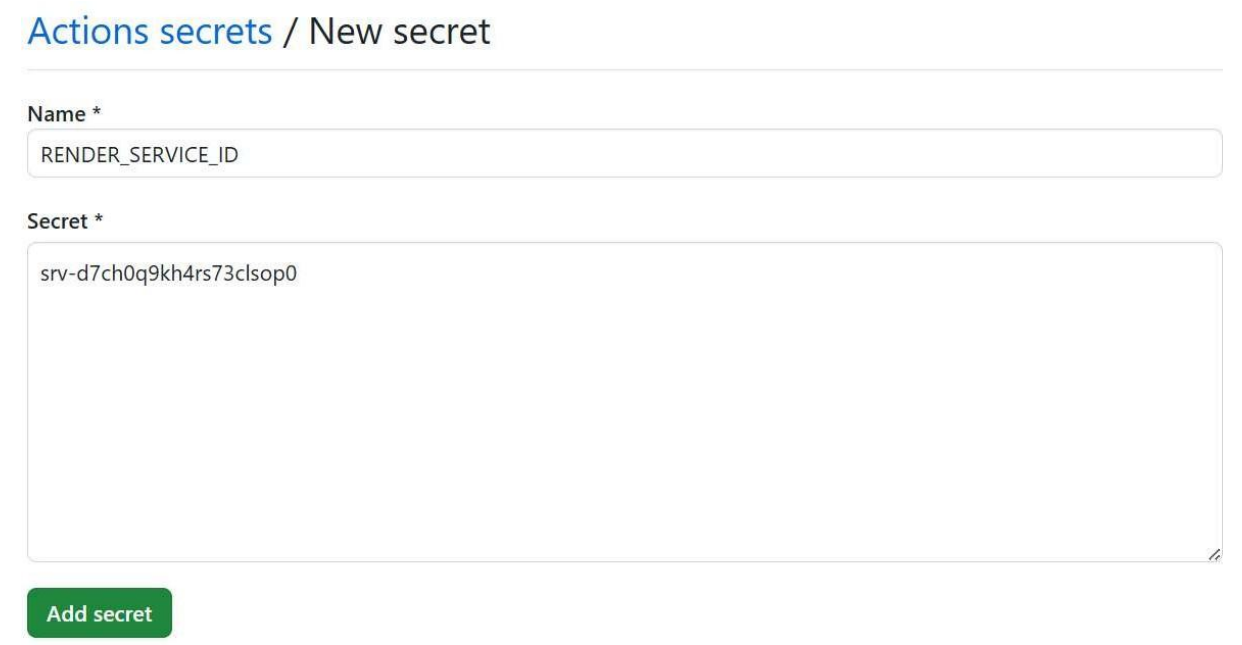

## 4.6 Procédure de déploiement :

### Configuration Render :




# Activation du CI/CD

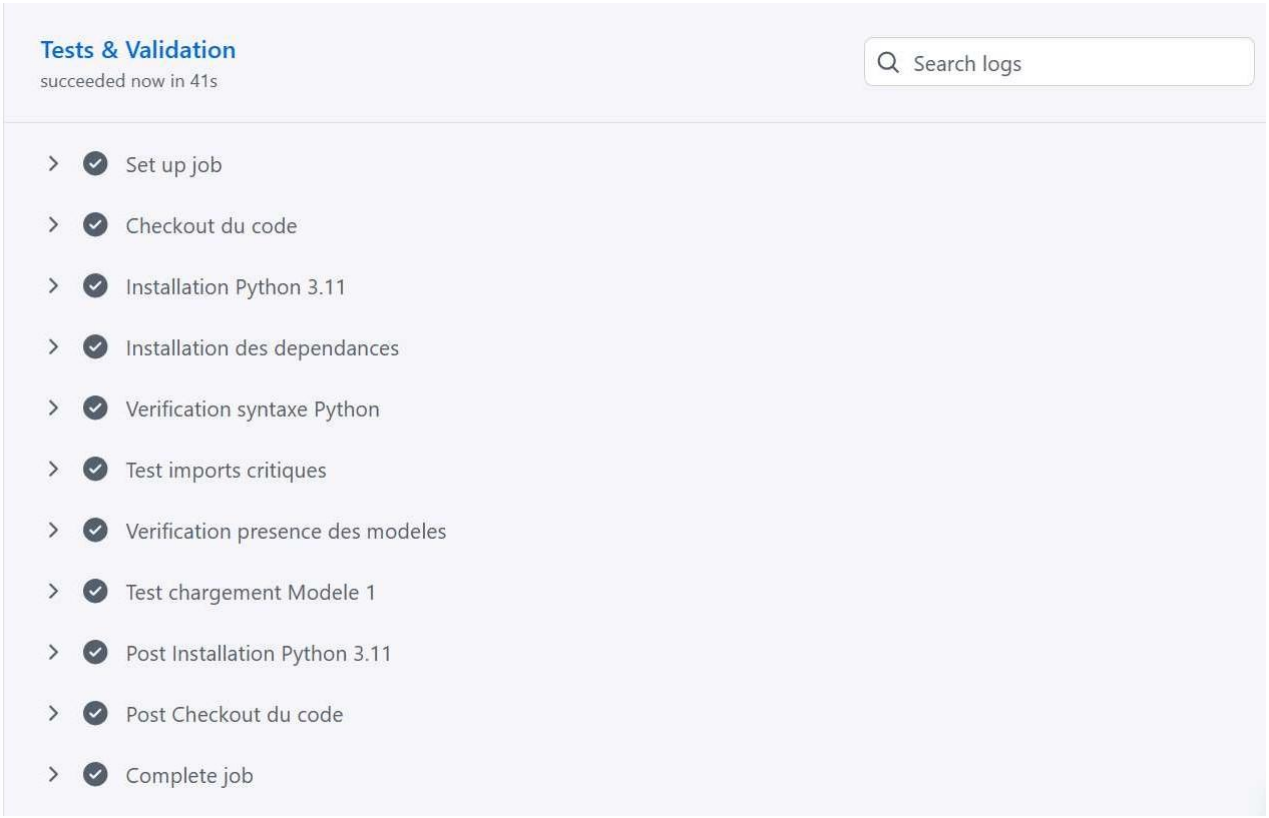


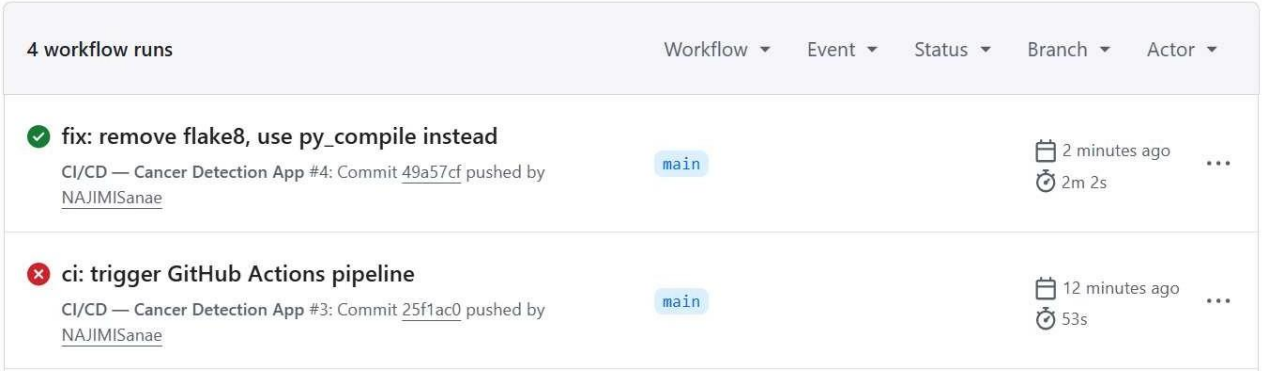




**Déclenchement des actions Github :**

```
>> git add README.md
>> git commit -m "ci: trigger GitHub Actions pipeline"
>> git push origin main
[main 0fe9325] ci: trigger GitHub Actions pipeline
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 README.md
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 325 bytes | 325.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0) pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/NAJIMISanae/cancer-detection-tp
   db64a34..0fe9325  main -> main
PS C:\Users\BARRAQ\Desktop\ML ESIC>
```

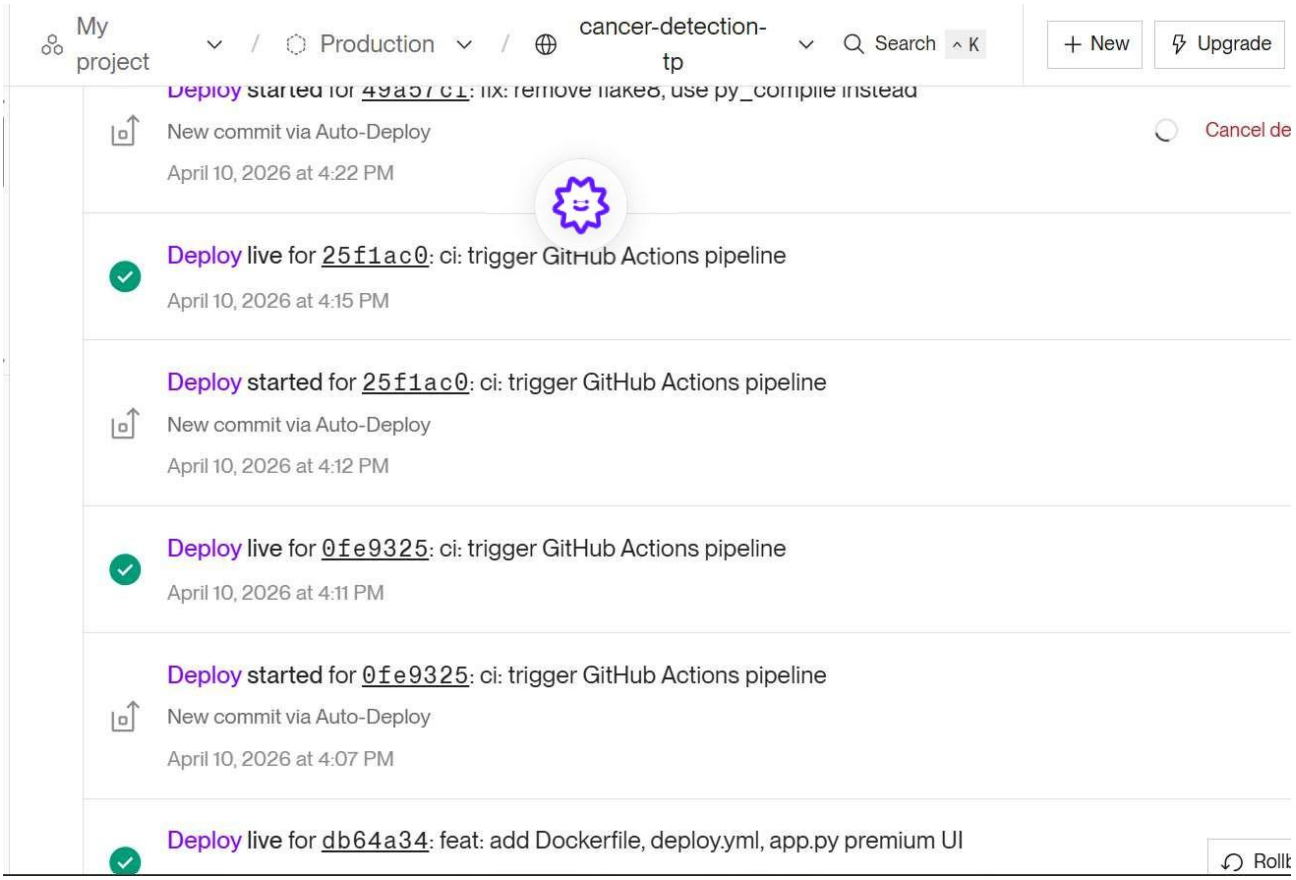
Tests avec succès :



**App déployée sur :**

https://cancer-detection-tp.onrender.com/




## 4.7 Flux CI/CD — de la modification de code au déploiement

Le diagramme suivant illustre le flux complet depuis la modification locale jusqu'à la mise à jour en production :

```
Développeur
  │
  ├─ Modifie app.py localement
  ├─ git add + git commit + git push origin main
  │
  ▼
GitHub
  │
  ├─ Détecte le push → déclenche .github/workflows/deploy.yml
  │
  ├─ Job 1 — TEST (ubuntu-latest)
  │   ├─ pip install -r requirements.txt
  │   ├─ flake8 app.py  (syntaxe)
  │   ├─ python -c 'import streamlit, joblib...'
```

```
│   └─ python test_model1.py  (predict_proba)
│
├─ Job 2 — BUILD (si test ✅)
│   ├─ docker build -t cancer-app:SHA .
│   └─ docker run --rm cancer-app:SHA python -c 'import streamlit'
│
└─ Job 3 — DEPLOY (si build ✅ ET push sur main)
    └─ curl POST api.render.com/v1/services/srv-xxx/deploys
                    │
                    ▼
                Render
                    ├─ Pull le code depuis GitHub
                    ├─ docker build (avec cache)
                    ├─ docker run → app disponible
                    └─https://cancer-detection-tp.onrender.com/
```

